

Revelator: Rapid Data Fetching via System-Software-Guided Hash-based Speculative Address Translation

Konstantinos Kanellopoulos¹ Konstantinos Sgouras¹ Harsh Songara¹
 Andreas Kosmas Kakolyris¹ Vlad-Petru Nitu¹ Spiros Galanopoulos¹ Rahul Bera¹
 Konstantina Koliogeorgi¹ Rakesh Kumar² Onur Mutlu¹

¹ETH Zürich ²NTNU

Address translation is a significant performance bottleneck in modern computing systems. Predicting the physical address (PA) of the requested data before address translation completes is a promising technique to hide the latency of address translation. However, accurately predicting the PA based on the virtual address (VA) is highly challenging due to the inherent unpredictability of the VA-to-PA mappings that conventional operating systems introduce. Prior works try to introduce predictability into VA-to-PA mappings but exhibit two key shortcomings: (i) they rely on the availability of large pages or VA-to-PA contiguity, and (ii) they store speculation-related metadata in costly hardware components whose power and area overheads may not always justify their benefits.

We introduce Revelator, a new hardware-OS cooperative technique that uses hashing to enable highly accurate speculative address translation with small system modifications. At the OS level, Revelator employs a tiered hash-based memory allocation policy for both program data and last-level page table entries (PTEs) to establish predictable VA-to-PA and VA-to-PTE mappings. At the hardware level, after an L2 TLB miss, a lightweight speculation engine uses the OS's hash functions to predict the VA-to-PA and VA-to-PTE mappings and prefetch the corresponding cache blocks before address translation completes, thereby both hiding address translation latency and accelerating page table walks (PTWs). Revelator comes with two key benefits: (i) it does not rely on large pages or VA-to-PA contiguity to achieve high speculation accuracy, and (ii) it requires small OS and hardware modifications.

Our evaluation across 11 data-intensive workloads shows that, (i) in single-core systems, Revelator provides an average speedup of 15.3% over the state-of-the-art speculative address translation technique under high memory fragmentation, and (ii) in virtualized environments, by accurately predicting both guest and host physical addresses, Revelator provides a 13.6% average speedup over Nested Paging. In multicore systems, Revelator's benefits scale with core count, enabling a speedup of $1.40\times$ ($1.50\times$) over Transparent Huge Pages (THP) across 30 server workload mixes from Google in a 16-core system under medium (high) memory fragmentation. These gains come at very small hardware cost. Our RTL synthesis shows that Revelator incurs only 0.02% area and 0.03% power overheads on top of a high-end server-grade CPU. Revelator is freely available at github.com/CMU-SAFARI/Virtuoso.

1. Introduction

As modern applications become increasingly data-intensive, address translation has emerged as a major performance bottleneck [1–14]. Recent studies from academia and industry demonstrate that address translation can account for

up to 40-45% of total execution time (15-20% with large pages) [1, 2, 5, 6, 14–23]. Address translation overheads are projected to grow as architectures transition toward larger physical address spaces, driven by paradigms like disaggregated memory [24–48], tiered memory systems [49–65], and unified virtual memory (UVM) in heterogeneous architectures [66–87].

Speculative Address Translation. Speculation is a promising technique to mitigate address translation overheads. By predicting the physical address (PA) corresponding to a virtual address (VA) before resolving the VA-to-PA mapping, the processor can begin fetching data using the PA immediately, without waiting for address translation to complete. This allows the speculative memory data access to overlap with the ongoing translation lookaside buffer (TLB) lookup and page table walk, reducing or hiding address translation latency. However, accurately predicting the PA from the VA remains highly challenging due to the inherent unpredictability of the VA-to-PA mappings that conventional OSes introduce. Because conventional OSes flexibly map virtual pages to *any* available physical frame, the exact mapping depends heavily on dynamic system conditions such as (i) memory fragmentation and (ii) memory allocation history.

Shortcomings of Prior Works. Prior works on speculative address translation [21, 88–90] attempt to form predictable VA-to-PA mappings. However, they rely on strong assumptions that limit their applicability and effectiveness in real systems. First, they rely on the availability of free large pages (e.g., 2MB) or VA-to-PA contiguity, neither of which can be guaranteed in practice, especially in systems with highly-fragmented memory (e.g., datacenters) or when multiple applications compete for memory [91, 92]. Second, they require significant changes to the processor architecture, including new power- and area-hungry hardware components that store speculation-related metadata. For example, Barr et al. [88] propose SpecTLB, a hardware TLB that opportunistically stores VA-to-PA mappings for reserved but not-yet-promoted large pages. However, as shown in §2.3, such schemes provide low benefits even under modest memory fragmentation.

Our goal is to design a speculation mechanism that hides address translation latency in modern systems without relying on large pages, VA-to-PA contiguity, or costly hardware structures to store speculation metadata. We introduce *Revelator*, a new hardware-OS cooperative scheme that uses hashing to enable highly accurate speculative address translation with small system modifications. **The key idea** of Revelator is to have the OS establish a *predictable* hash-based virtual-to-physical mapping, which the hardware exploits to perform highly accurate speculative address translation to predict which data to fetch. The key benefits of Revelator are twofold: (1) its prediction

accuracy is not affected by memory fragmentation, as *per-page* hash-based allocation succeeds with a probability that depends *only* on memory utilization and *not* on how free frames are distributed (e.g., available free large pages, VA-to-PA contiguity), and (2) it requires small OS and hardware modifications, i.e., minor extensions to the buddy allocator to enable hash-based data allocation and a lightweight speculation engine that generates candidate PAs based on the same hash functions used by the OS.

Key Mechanism. At the OS level, Revelator employs a *tiered* hash-based allocation policy to increase the likelihood of establishing a predictable VA-to-PA mapping. When allocating a physical frame for a virtual page, the OS performs up to N hash-based allocation attempts (i.e., allocation tiers). In each attempt i , the OS applies the same hash function to the virtual page number (VPN) with a tier-specific seed, yielding a candidate physical page number (PPN_i). The OS allocates the first PPN_i that corresponds to a free physical frame, falling back to the conventional allocator only if all N candidates are already allocated. Because each hash-based allocation attempt acts as an independent trial, the probability that all N candidates are already occupied decreases exponentially with N , ensuring a high success rate for establishing predictable VA-to-PA mappings even under high memory utilization.

At the hardware level, on an L2 TLB miss, a lightweight speculation engine first recomputes the same N hash-derived candidates and combines each candidate PPN_i with the page offset, producing N candidate PAs. Revelator does not blindly issue speculative accesses to all N candidate PAs. Instead, a speculation degree filter selects how many candidates to fetch based on the recent usefulness of each allocation tier and the current memory bandwidth pressure. Thus, the speculation engine issues at most N speculative data requests, and often fewer depending on system conditions. If one of the candidates is correct, the data can arrive before the PTW completes, effectively hiding address translation latency.

Revelator applies the same hash-based allocation and speculation to the page table (PT) itself. Similar to data page allocation, the OS attempts to place each last-level PT frame at $PPN_i = Hash_i(VPN \gg 9)$, where the 9-bit shift accounts for the 512 VPNs that share each PT frame. On an L2 TLB miss, the hardware speculatively fetches this last-level PT frame in parallel with the earlier levels of the walk. Altogether, Revelator achieves two levels of concurrency: (1) it overlaps the final data fetch with the entire page table walk, and (2) it overlaps the fetch of the last-level PT with earlier stages of the walk itself.

Integration with Large Pages and NUMA. Revelator fully supports large-page allocation and NUMA-aware placement, two widely deployed features of modern OSes. For large pages, Revelator+THP combines the TLB-reach benefits of Transparent Huge Pages (THP) [93, 94] when 2MB pages are available with Revelator’s hash-based 4KB speculation when fragmentation makes large pages unavailable, enabling high performance across the fragmentation spectrum (§4.4). For NUMA systems, Revelator preserves the OS’s node-selection policy and introduces a new lightweight, Bloom-filter-based data structure that tracks which NUMA node each virtual region maps to, enabling the hardware to speculate accurately even when a thread accesses pages on remote NUMA nodes (§4.6).

Evaluation Methodology. We prototyped Revelator’s

tiered hash-based allocation mechanism in the Linux kernel (v.6.10.8) [95] to demonstrate its feasibility and effectiveness. We evaluate the effectiveness of Revelator in single-core, multi-core, and virtualized environments using Virtuoso [13, 96], a validated simulation methodology that enables high-accuracy full-system simulation ported on top of Sniper [97, 98], an event-driven multi-core simulator. In single-core systems, we evaluate Revelator across 11 translation-intensive benchmarks from GraphBIG [99], GUPS [100], DLRM [101], GenomicsBench [102], and XSBench [103]. In multi-core systems, we evaluate Revelator on 30 mixes from 140 Google Server Traces [104]. We compare Revelator against eight state-of-the-art address translation schemes [16, 19, 21, 88, 93, 105–108] and a system with a very large 64K-entry L2 TLB.

Key Results. Our evaluation yields seven key results that show Revelator’s effectiveness. First, our Linux prototype, running on a real high-end system, shows that Revelator’s tiered hash-based allocator using only three hash functions successfully allocates more than 70% of pages even under high memory utilization levels (Fig. 11), with minimal overhead (0.078% on average) on total execution time. Second, in single-core systems, Revelator provides a 19% (25%) average speedup over THP, and an 8.5% (15.3%) average speedup over SpOT [21], the state-of-the-art speculative address translation technique, under medium (high) memory fragmentation. Revelator provides these gains while minimizing: (1) the number of speculative L2 data cache fills that lead to harmful evictions and (2) the bandwidth spent on unused speculative data fetches. Third, in virtualized environments, Revelator provides a 13.6% average speedup over Nested Paging [109] with THP [93] (NP+THP) by accurately predicting both guest and host physical addresses. Fourth, in multi-core systems, Revelator’s benefits scale with core count, enabling an average speedup of $1.40\times$ ($1.50\times$) over THP at 16 cores under medium (high) memory fragmentation, while sustaining 87–88% speculation accuracy across all core counts. Fifth, in an 8-node/64-core NUMA system, Revelator outperforms SpOT by up to $1.20\times$. Sixth, Revelator+THP reduces energy consumption by 5.5% over THP in a system with medium fragmentation, and by 2.2% over Mosaic Pages [106], a scheme with very large TLB reach. Seventh, our RTL synthesis results show that Revelator incurs only 0.02% area and 0.03% power overheads on top of a high-end server-grade CPU [110].

In this work, we make the following contributions:

- We introduce *Revelator*, a new hardware-OS cooperative scheme that uses predictable hashing to enable highly accurate speculative address translation with minimal system modifications. **The key idea** of Revelator is to have the OS establish a predictable hash-based virtual-to-physical mapping, which the hardware exploits to perform highly accurate speculation. This both hides and reduces address translation latency and provides two key benefits: (i) Revelator is not affected by memory fragmentation and (ii) it requires very small OS and hardware modifications.
- We prototype Revelator’s allocator in the Linux kernel (v.6.10.8), evaluate Revelator via simulation across diverse benchmarks against nine prior techniques, and validate its hardware costs via RTL synthesis. Our results demonstrate significant performance and energy gains in single-core, multi-core, NUMA, and virtualized systems compared to all prior techniques. Revelator is freely available at github.com/CMU-SAFARI/Virtuoso.

2. Motivation

2.1. Speculative Address Translation

Previous works [21, 88–90, 111] hide address translation latency by speculating on the physical address, allowing the processor to issue early memory accesses and fetch data before address translation completes: a process called *speculative address translation*. Figure 1 shows the data fetch sequence under conventional and speculative address translation. In a conventional system, address translation and data fetching happen sequentially. The processor first performs a TLB lookup to translate the virtual address (VA) to a physical address (PA). If the TLB lookup results in a miss, the processor initiates a page table walk (PTW) to resolve the VA-to-PA mapping. Only after the PTW completes can the processor issue the memory access to fetch the data. This sequential process can lead to significant performance degradation, especially when the PTW requires high-latency DRAM accesses to complete. With speculative address translation, the processor predicts the physical address and issues an early speculative memory access to fetch the data before starting the PTW. If the prediction is correct, the speculative fetch can partially or fully overlap data fetching with address translation, hiding the latency of address translation and improving performance. If the prediction is incorrect, the processor fetches an irrelevant cache block. Such mispredictions do not affect correctness, but consume memory bandwidth, waste energy, and may pollute the cache.

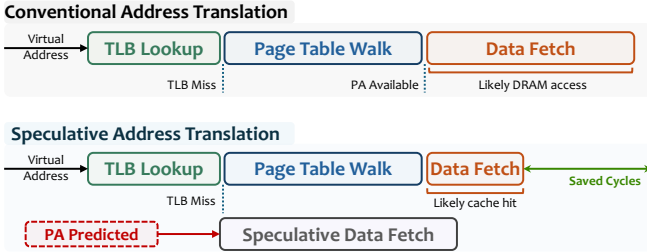


Figure 1: Data fetch sequence under (top) conventional address translation and (bottom) speculative address translation.

2.2. Challenges in Speculative Address Translation

Predicting the physical location of data based on the virtual address is challenging because conventional OSes do not establish stable, easily predictable VA-to-PA mappings.¹ First, modern OSes use fully-associative virtual memory: during a page fault, the allocator maps the faulting virtual page to any currently available physical frame [112]. Second, memory fragmentation and memory utilization continually change which frames are available. Together, these effects make VA-to-PA mappings highly dependent on runtime conditions, limiting the potential for accurate PA prediction.

2.3. Limitations of Prior Works

SpecTLB [88], the first work that proposed speculative address translation, relies on the OS to reserve large pages in order to achieve accurate hardware-side PA prediction. On the OS

side, a FreeBSD-like [105] OS allocation scheme reserves candidate 2MB regions but does not immediately use them as large pages. Instead, it allocates 4KB pages from the reserved region and only promotes the region to a large page when the OS detects that the region is highly utilized. On the hardware side, SpecTLB adds a TLB structure that records reserved but not-yet-promoted large-page regions and predicts the PA by speculatively assuming a fixed VA-to-PA offset within each reserved region.

SpOT [21] extends this idea from fixed-size large pages to arbitrarily-large contiguous virtual-to-physical mappings. On the OS side, SpOT uses contiguity-aware paging (CA-paging) to sustain contiguous VA-to-PA mappings over time. Whenever possible, CA-paging places pages from the same virtually contiguous memory object, called a Virtual Memory Area (VMA) in Linux [113], in a contiguous physical segment, while preserving standard demand paging (i.e., pages are allocated on demand). On the first fault to a VMA, CA-paging searches a contiguity map, an OS metadata structure that tracks available contiguous physical segments, to find a free physical segment that can accommodate the VMA. It then allocates the faulting page in that segment, records the resulting VA→PA offset for the VMA (i.e., the fixed PA-minus-VA displacement implied by that first placement), and uses that offset as a guide to allocate later-faulted pages from the same VMA in a best-effort manner. This way, CA-paging tries to establish contiguous VA-to-PA mappings over time, which SpOT leverages to perform accurate speculative address translation. On the hardware side, SpOT leverages the contiguous mappings created by CA-paging and caches them as contiguity descriptors in a small prediction table indexed by the load/store program counter (PC). Each descriptor records the virtual range (base and bounds), access permissions, and a speculative VA→PA offset for a recently observed mapping. On a TLB miss, the processor uses the load/store PC to retrieve a contiguity descriptor, checks whether the missed VA falls within the descriptor’s range, and computes a speculative physical address as VA + offset.

2.3.1. Limits of Contiguous Multi-Page Mappings. Table 1 shows the key properties of contiguity-based speculative address translation techniques that rely on (i) structured contiguity (e.g., SpecTLB [88]) or (ii) arbitrary contiguity (e.g., SpOT [21]). Both SpecTLB [88] and SpOT [21] depend on preserving *multi-page invariants*: structured and fixed-size multi-page physical regions (e.g., $512 \times 4\text{KB}$ pages) in SpecTLB [88] or arbitrarily-large (i.e., base and bounds of a single mapping can be arbitrary) contiguous physical segments in SpOT [21]. These invariants are challenging to maintain when (i) memory fragmentation, which is observed to be high in mobile and large-scale systems (e.g., datacenters) [5, 92], severely limits the available physical memory contiguity, and (ii) co-running applications compete for contiguous allocations, a phenomenon we call *allocation interference*, which can rapidly increase memory fragmentation even when the system is lightly loaded.

We quantitatively analyze the sensitivity of contiguity-based speculative address translation techniques to fragmentation and allocation interference, which motivates Revelator’s hash-based placement invariant. We focus on SpOT [21] as it is the most recent work that relies on contiguity, but our analysis applies to any approach that relies on contiguous multi-page mappings for speculation (e.g., SpecTLB [88]).

¹Predicting PAs using other program features (e.g., PCs), beyond VAs, is not meaningful because in conventional OSes the faulting VA and its properties (e.g., access type, permissions) are the only relevant program-visible features that affect PA selection.

Table 1: Comparison between (i) contiguity-based and (ii) Revelator’s hash-based speculative address translation techniques.

Property	Structured contiguity (SpecTLB [88])	Arbitrary contiguity (SpOT [21])	Hash-based allocation (This work: Revelator)
Placement invariant	Fixed-size 2MB pages	Arbitrarily-large contiguous physical segments	Per-page hash-based placement
Speculation metadata	None	Contiguity descriptors per memory object	Counters per hash function
Hardware complexity	▼ Extra TLB for reserved large pages	▼ Cache for contiguity descriptors	▲ Hash function circuitry and counters
Sensitivity to fragmentation	▼ Single "hole" prevents reserving a large page*	▲ Speculation coverage and accuracy drops as "holes" increase	▲ Successful hash-based allocation depends only on memory utilization
Allocation interference	▼ Allocation interference creates holes	▼ Allocation interference disrupts contiguity	▲ Independent of allocation interference
Memory allocation	▲ Fewer physical memory allocations	• Minimal impact on latency	▼ Small increase in latency

* A "hole" is an occupied page within an otherwise contiguous physical region that splits a large block into smaller non-contiguous segments, creating memory fragmentation.

2.3.2. Sensitivity to Fragmentation. To quantify SpOT’s sensitivity to fragmentation, Figure 2 shows the average speculation coverage (i.e., the fraction of TLB misses that are speculatively translated) and accuracy (i.e., the fraction of correct speculative address translations) across 11 translation-intensive workloads at different fragmentation levels (see §6 for details). We make three key observations. (1) Under low fragmentation (<1%), SpOT delivers near-perfect accuracy ($\approx 100\%$) and high coverage ($>85\%$). (2) At medium fragmentation levels (10–20%), accuracy drops to $\approx 70\%$ and coverage degrades sharply: at 10% fragmentation, coverage reduces to $\approx 40\%$, and beyond 20% fragmentation, SpOT speculates on fewer than 20% of TLB misses. (3) At high fragmentation ($\approx 50\%$), SpOT provides very low coverage (below 10%). Together, these observations show that the effectiveness of contiguity-based speculative address translation techniques (e.g., SpOT) heavily depends on memory fragmentation. In §7.1, we further show the performance gains provided by SpOT and Revelator under different fragmentation levels over the baseline system.

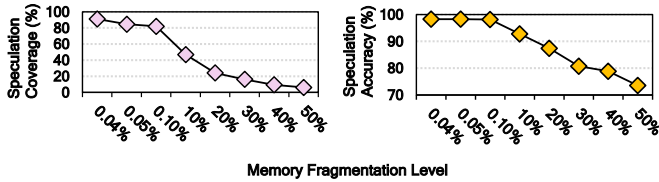


Figure 2: Speculation coverage (left) and accuracy (right) of SpOT [21], a state-of-the-art speculative address translation technique, in a system with different memory fragmentation levels.

2.3.3. Sensitivity to allocation interference. SpOT’s data placement strategy is effective when a small number of VMAs can be mapped to large contiguous physical segments, but it becomes fragile under allocation interference. Since SpOT does not preallocate the entire VMA, other VMAs or co-running applications can consume pages inside the targeted physical segment before subsequent page faults occur. Interleaved allocations create holes, disrupt the contiguity that SpOT relies on for speculative translation, and force the system to fall back to the conventional allocation policy (i.e., fully-associative data placement).

Figure 3 quantifies how allocation interference affects (i) the available physical memory contiguity and (ii) the ratio of successful offset-based allocations, i.e., the fraction of allocations that follow the VMA’s recorded VA→PA offset. In this experiment, (1) memory is minimally fragmented (i.e., memory utilization is less than 1%) to isolate the effects of allocation in-

terference, (2) we use a microbenchmark that allocates memory for a configurable number of 1GB VMAs, and (3) we execute applications in a simulated system that employs a round-robin thread scheduler and 128GB of main memory (see §6 for details about the simulation methodology).

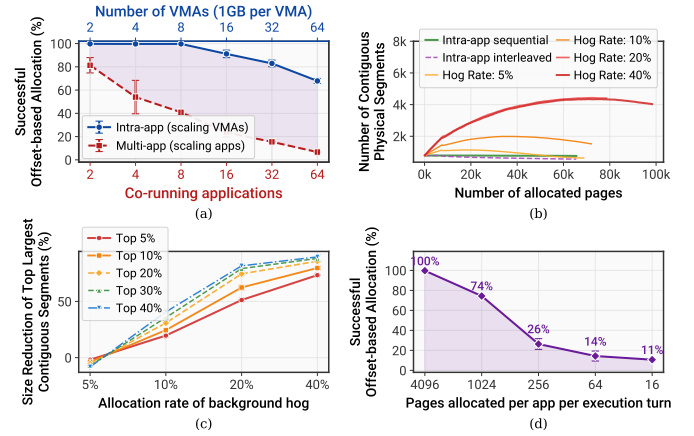


Figure 3: Sensitivity of SpOT to allocation interference: (a) ratio of successful offset-based allocations as the number of VMAs or co-running applications increases, (b) size reduction of the largest contiguous physical segments under background memory hogging, (c) increase in the number of contiguous physical segments under background memory hogging, and (d) successful offset-based allocations under finer-grained allocation interleaving.

Figure 3 (a) shows the ratio of successful offset-based allocations (i) as the number of VMAs within one application increases and (ii) as the number of co-running applications increases. We make two key observations. First, increasing the number of VMAs within a single application gradually reduces the ratio of successful offset-based allocations. The success rate remains near 100% with up to 8 VMAs and stays above 60% even at 64 VMAs. Second, increasing the number of co-running applications reduces the ratio of successful offset-based allocations from $\approx 80\%$ with 2 applications to $\approx 5\text{--}10\%$ with 64 applications. We conclude that SpOT is more sensitive to inter-application interference than to the number of VMAs within a single application, because concurrent applications interleave independent allocation streams and create irregular allocation patterns that rapidly disrupt the available physical memory contiguity.

Figure 3 (b) shows the number of free contiguous physical segments for different hog rates as a foreground application runs alongside the background hog. We observe that as the hog

rate increases and as the foreground application allocates more memory, the number of free contiguous physical segments grows from a few dozen to several thousand. This growth in the number of segments indicates that the free physical memory becomes highly fragmented, thereby reducing SpOT’s effectiveness (as shown in §7.1).

Figure 3 (c) measures how background memory hogging (i.e., allocation and freeing of pages performed by a co-running background process) affects the largest available contiguous physical segments. In this experiment, we run a background memory hog that allocates and frees pages at different rates (i.e., hog rate of $N\%$ means that the background hog allocates N pages per 100 pages allocated by the foreground application) alongside the foreground application. We make two key observations. First, even a low hog rate (20%) reduces the largest contiguous physical segments by 50–80%. Second, as the hog rate increases, the largest contiguous physical segments degrade further, and at 80% hog rate, the largest contiguous segment consists of only a few pages (≈ 32). We conclude that even moderate background allocation activity can break the large free contiguous physical segments into substantially smaller ones, which in turn significantly reduces SpOT’s speculation coverage and accuracy (as shown in §7.1).

Figure 3 (d) shows the effect of allocation interleaving granularity with 16 applications running concurrently. We vary the allocation turn size (i.e., the number of pages each application allocates before the OS scheduler switches execution to the next application) from 4096 pages down to 16 pages. We observe that reducing the allocation turn size from 4096 pages to 16 pages reduces successful offset-based allocation from 100% to 11%. A smaller allocation turn size interleaves allocation requests from different applications more frequently, creating holes inside the physical segments that SpOT tries to preserve. We conclude that SpOT’s offset-based placement breaks down rapidly under fine-grained allocation interleaving.

Taken together, these results show that SpOT’s reliance on contiguous physical regions makes speculation fragile under realistic allocation dynamics: competing VMAs, co-running applications, and background allocation activity all break large free regions into smaller pieces, reducing offset-based allocation success. Our goal in this work is to design a speculative address translation mechanism that does *not* rely on contiguous multi-page mappings and can instead provide high speculation coverage and accuracy even under high fragmentation and high loads of allocation interference.

3. Revelator: Design Overview

We present Revelator, a hardware-OS cooperative scheme that uses hashing to enable highly accurate speculative address translation with minimal system modifications. The key idea of Revelator is to have the OS establish a predictable per-page hash-based data placement that the hardware leverages to predict the physical address of requested data. Figure 4 shows an overview of Revelator’s key components and workflow.

Key Mechanism. At the OS level, Revelator employs a *tiered* hash-based allocation policy. When the OS receives a request to allocate a physical frame for a virtual page ①, it computes a bounded sequence of candidate physical page numbers (PPNs), with one candidate per allocation tier (i.e., hash attempt). For tier $i \in [1, N]$, the OS applies the same hash function h to the VPN and a tier-specific seed $seed_i$, producing

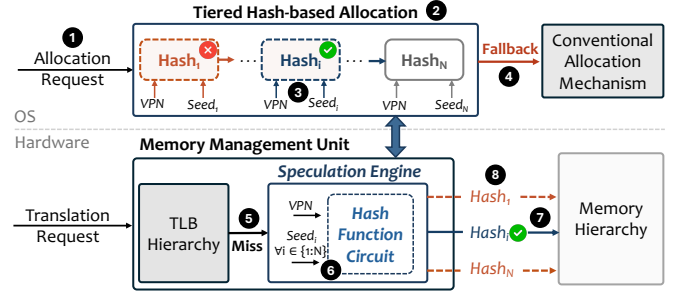


Figure 4: Revelator Overview

$PPN_i = h(VPN, seed_i)$. The OS attempts to allocate the virtual page to these candidate frames in tier order ②. If PPN_i is free, the OS allocates the virtual page to the corresponding physical frame ③, establishing a predictable VA-to-PA mapping that the hardware can later recompute using the same VPN and seed. If PPN_i is occupied, the OS advances to the next tier and computes another deterministic candidate using the same VPN but a different seed. Because each tier probes an independently chosen candidate frame, a system with memory utilization u gives each candidate an approximately $1 - u$ chance of being free, making it increasingly unlikely that all N candidates are occupied, even under high memory utilization. If all N candidate physical frames are occupied, the OS falls back to the conventional allocation policy ④. We analyze the probability of successful hash-based allocation and its dependence on memory utilization in Section 4.1.

At the hardware level, when a translation request misses in the TLB hierarchy ⑤, Revelator invokes a lightweight speculation engine that mirrors the OS’s tiered hash-based allocation policy. First, the speculation engine uses the same hash function circuit and tier-specific seeds to recompute the candidate $PPNs$ for the miss-causing VPN ⑥. Second, it combines each candidate PPN_i with the page offset to form a candidate physical address and issues selected speculative data requests to the memory hierarchy. If a candidate corresponds to the actual physical address ⑦, the speculative data fetch overlaps with the page table walk, and the data can arrive before address translation completes. In the example of Figure 4, the OS successfully allocates the virtual page to the candidate frame in tier i . However, the hardware does not know a priori which tier the OS used to place the data, so it may also issue extra speculative fetches for candidates from other tiers, such as tiers 1 and N ⑧. In §4.3, we describe how the speculation engine uses a speculation degree filter to reduce this extra memory traffic by limiting which candidate requests are issued.

Key Benefits. As shown in Table 1, Revelator avoids the limitations of prior contiguity-based mechanisms. First, Revelator’s prediction accuracy does *not* depend on memory fragmentation but only on memory utilization. This is because Revelator’s allocation policy enforces a *per-page* hash-based placement invariant, rather than relying on free contiguous multi-page physical segments. Second, as Revelator’s effectiveness does not depend on memory fragmentation, it is naturally resilient to allocation interference. Third, Revelator requires only small OS changes: (i) the OS allocator is modified to support tiered hash-based allocation, and (ii) the OS maintains a small amount of counters per hash function to track the successful allocation rate of each hash function and guide the

hardware-based speculation. Fourth, Revelator requires only small hardware changes: (i) a lightweight speculation engine that computes the same hash function as the OS and (ii) a small number of hardware counters to track the successful allocation rate per hash attempt and guide speculation.

4. Revelator: Detailed Design

We describe in detail the key components of Revelator’s design, including: (i) tiered hash-based allocation policy for data pages and its probabilistic properties, (ii) rationale behind per-page placement instead of contiguous multi-page placement, (iii) extension of tiered hash-based allocation to page table frames, (iv) speculation engine and how it dynamically adapts the degree of speculation, and (v) how Revelator operates in the presence of large pages, in virtualized environments, and NUMA systems. The extended version of this paper [114] provides a detailed example of Revelator’s end-to-end flow of memory allocation and address translation.

4.1. OS: Tiered Hash-Based Allocation

Revelator introduces a tiered hash-based physical page allocation policy that establishes a predictable per-page mapping from virtual page numbers (VPNs) to physical page numbers (PPNs). For each allocation, the OS computes a bounded sequence of candidate PPNs, with one candidate per allocation tier. We use the terms *tier* and *hash attempt* interchangeably to refer to one ordered step in this sequence: tier i uses the same hash function h but a distinct tier-specific seed $seed_i$, producing $PPN_i = h(VPN, seed_i)$. This tiered candidate set narrows the possible physical locations for each VPN, enabling hardware to later recompute the same candidates and speculate on the most likely physical addresses.

Hash-based Allocation Attempt. Upon an allocation request, the OS starts at tier 1 and computes $PPN_1 = h(VPN, seed_1)$. It then checks whether PPN_1 is a valid candidate frame and is present in the buddy allocator’s free lists. If the frame is free, the OS removes it from the buddy allocator and maps VPN to PPN_1 . If the frame is occupied, reserved, or otherwise unavailable, the OS advances to the next tier and computes another deterministic candidate using the same VPN but a different seed. This process continues for tiers $2, 3, \dots, N$ until either a free candidate PPN is found or all N hash attempts are exhausted. If all N candidate PPNs are unavailable, Revelator falls back to the conventional allocation mechanism to find any available free page PPN_{FB} . In this case, the VPN to PPN_{FB} mapping is not predictable.

Probabilistic Analysis. Assuming each hash attempt H_i produces candidate PPNs that are uniformly distributed across the physical address space, each hash-based allocation attempt succeeds with a probability of $p = M/P$, where M is the number of free memory pages and P is the total number of memory pages. Revelator’s tiered hash-based allocation mechanism can be modelled as N sequential Bernoulli trials. Thus, the probability that at least one of the N hash attempts succeeds is $P_S = \sum_{i=0}^{N-1} p(1-p)^i = 1 - (1-p)^N$. As N increases, the probability that all candidates are unavailable decreases exponentially as $(1-p)^N$. Our Linux kernel prototype of Revelator validates the exponential decrease (§5.2). Even though increasing N improves the probability of a successful hash-based allocation, it also increases (i) the number of hash-based allocation attempts the OS may perform during allocation and (ii) the

number of candidate physical addresses the hardware needs to fetch before translation completes, potentially increasing allocation latency and memory bandwidth consumption.

Leveraging Tiering-Induced Bias. The probability that allocation is successful at hash attempt i decreases geometrically in i : $P(Alloc_i) = p(1-p)^{i-1}$ (with $i=1$ corresponding to H_1). This induces a *tiering bias*: earlier hash attempts produce more allocations than later ones — H_1 produces the most, H_2 the next-most, and so on. Revelator’s speculation mechanism leverages this bias by generating candidate physical addresses in tier order and issuing the most likely speculative fetch first, i.e., the candidate derived from $H_1(VPN)$. Issuing additional fetches for more candidates such as $H_2(VPN)$ and $H_3(VPN)$ can improve speculation coverage, but it also consumes additional memory bandwidth (§4.3.2 describes how Revelator dynamically selects how many candidates to fetch).

Choice of Hash Function. Revelator uses CityHash [115], a fast non-cryptographic hash function also used in prior hash-based schemes [19, 116, 117]. The OS and hardware use the same hash function so that the hardware can deterministically recompute the candidate PPNs generated during allocation. To generate multiple candidates, Revelator feeds the hash function with the VPN, the process identifier (PID), and a tier-specific counter-based seed: $PPN_i = \text{CityHash}(VPN, PID, seed_i)$.²

4.2. Hash-Based Page Table Allocation

Revelator also applies hash-based allocation to final-level page table (PT) frames, making the VA-to-PTE mapping predictable and enabling the hardware to speculatively fetch the final PTE at the start of the PTW. Unlike data-page allocation, PT-frame allocation uses a single hash attempt rather than multiple tiers (a conservative choice because the potential performance benefit of additional PT-frame hash attempts does not justify the cost of issuing multiple speculative PTE fetches per TLB miss). Figure 5 illustrates Revelator’s allocation policy for PT frames. When allocating a final-level PT frame, the OS computes one target PPN using $H_1(VPN >> 9)$, where the VPN is right-shifted by 9 because each final-level PT frame contains PTEs for 512 contiguous VPNs ❶. If the target PPN is free, the OS allocates the PT frame to that PPN and updates the corresponding page-directory (PD) entry with the selected PT-frame PPN, allowing a conventional PTW to normally reach the PT frame ❷. Otherwise, Revelator falls back to the conventional allocator to find a frame for the PT. When the hash-based PT-frame allocation succeeds, the MMU can also recompute the same PT frame’s PPN from the VPN, combine it with the PTE offset, and speculatively fetch the final PTE at the start of the PTW ❸. This overlaps the final PTE fetch with the earlier levels of the PTW, accelerating the walk and reducing address translation latency (§4.3).

4.3. Hardware: OS-Guided Speculative Translation

4.3.1. Speculation Engine. The speculation engine is part of the memory management unit (MMU) and works in tandem with the OS’s tiered hash-based allocation policy to predict the physical page number (PPN) for a given virtual page number (VPN) and fetch the corresponding data before address translation completes.

²Using the PID as input to the hash function ensures that the same VPNs across different processes map to different candidate PPNs.

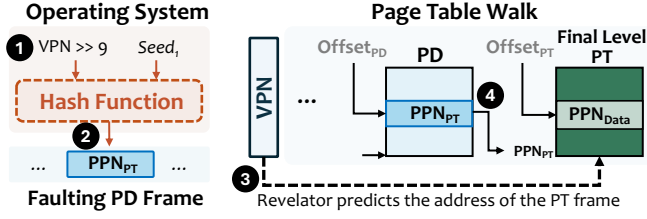


Figure 5: Revelator’s hash-based allocation for page table frames.

Speculative PA Calculation. On an L2 TLB miss, the speculation engine extracts the VPN and page offset from the faulting VA. It then recomputes the same bounded sequence of candidate PPNs that the OS considered during allocation by hashing the VPN with each configured seed. Each candidate PPN is concatenated with the page offset to form a candidate PA. For final-level page table frames, the speculation engine similarly hashes $VPN \gg 9$ to predict the PT-frame PPN and combines it with the PTE offset to form a candidate PTE address. Because the hardware does not know a priori which hash attempt succeeded during OS allocation, it can generate up to N candidate PAs, one per tier, and issue speculative fetches in the configured tier order. The latency of the hash function circuitry is only 2–3 cycles, which is negligible compared to the latency of a PTW (40–100 cycles).

4.3.2. Dynamically Adapting Speculation Degree. Revelator needs to decide how many of the hash-based candidate PPNs to turn into actual speculative memory requests on each L2 TLB miss, which we refer to as the *speculation degree*. The speculation degree controls a trade-off between address translation coverage and memory system pressure. A larger degree (e.g., $N=4$) covers pages allocated by later tiers and therefore increases the likelihood that at least one speculative fetch targets the correct PPN. However, for each additional tier, an additional speculative memory request needs to be issued, consuming bandwidth and potentially delaying demand accesses. A smaller degree (e.g., $N=1$) avoids unnecessary traffic when most pages are allocated by the first tier, but it misses pages that were allocated by later tiers ($N>1$), which become more common as memory utilization increases. Because the right degree depends on both allocation behavior and available bandwidth, Revelator uses a speculation degree filter to dynamically select how many candidate PAs to fetch.

Memory Utilization: As memory utilization increases, the OS is more likely to use secondary tiers (H_2 , H_3 , etc.) or the conventional allocator to allocate a page. In such scenarios, increasing the speculation degree improves speculative hit likelihood.

Memory bandwidth: When memory bandwidth is abundant, Revelator can speculate on more candidate PAs because the extra requests are less likely to contend and safely improve speculation coverage without hurting demand requests. When bandwidth is scarce, however, speculative requests to candidate PAs that do not match the OS-selected PPN become costly: they occupy memory queues, consume interconnect and DRAM bandwidth, and can delay demand requests.

The Speculation Degree Filter: Figure 6 shows the design of the speculation degree filter. It employs two monitors to dynamically adapt speculation degree. First, the memory utilization monitor tracks the fraction of all successful allocations

performed by each tier. Second, the bandwidth monitor tracks bandwidth usage. The speculation degree filter uses the utilization monitor to drop tiers that have allocated fewer pages than a pre-defined threshold. For the remaining tiers, the filter adaptively adjusts the degree based on the available memory bandwidth. The final PA candidates are speculatively fetched.

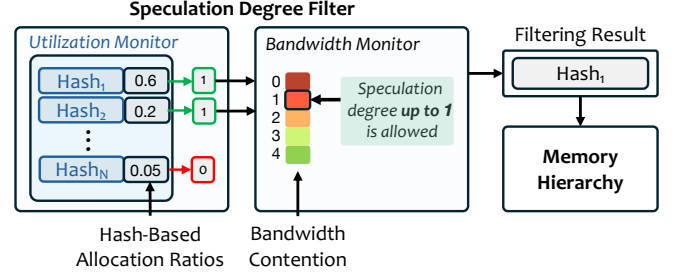


Figure 6: Speculation degree filter design.

4.4. Synergy with Large Pages

Large pages reduce the overhead of address translation when the OS can find sufficiently large contiguous physical regions, but their benefits degrade in the presence of fragmentation. Revelator addresses this limitation by making the physical location of 4KB pages predictable through hash-based placement, while still opportunistically exploiting larger regions when they are available. This lets Revelator preserve fast translation under fragmentation without giving up the benefits of large pages in low-fragmentation settings.

Per-Page vs. Multi-Page Hash-based Placement. A natural design question for Revelator is whether hash-based allocation should remain strictly per-page, or instead try to preserve predictable placement for larger multi-page regions, similar in spirit to mechanisms that support multiple page sizes [118–121]. Such a design would map k consecutive virtual pages to a hash-derived k -page physical region, allowing hardware to speculate at a coarser granularity. The drawback is that predictability now requires contiguity: allocation succeeds only if physical memory contains a sufficiently large contiguous region of free pages, so success depends on fragmentation, not just total free capacity.

Figure 7 shows the probability of successfully allocating k consecutive pages using a hash-based allocator with 1–4 allocation tiers (i.e., independent hash attempts). This experiment was conducted using the simulation infrastructure discussed in §6, by running a custom allocation-heavy microbenchmark. We use CityHash [115] as the underlying hash function and during allocation we derive each candidate physical range by hashing the virtual page number at k -page granularity (i.e., computing $CityHash(VPN \gg \log_2 k)$), where k ranges from 1 to 10. All experiments use identical memory utilization levels (40% of memory is allocated) but different fragmentation levels.

We make two key observations. First, increasing the number of tiers increases the allocation success probability, because each additional tier provides another independent chance to find a free region. Second, successful allocations reduce fast as k increases when memory fragmentation is medium/low, even when multiple independent hash attempts are available. In contrast, when $k = 1$, allocation success depends only on the fraction of free pages and is independent of fragmentation structure. This observation motivates Revelator’s core design

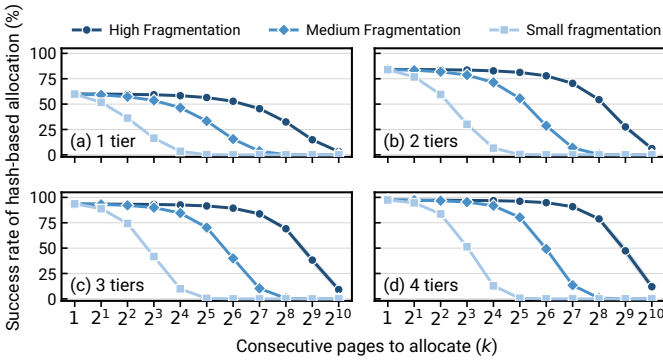


Figure 7: Ratio of successful hash-based allocations for k consecutive pages under different fragmentation patterns and numbers of hash attempts (i.e., tiers) in a 40% utilized memory.

choice to enforce a per-page placement ($k = 1$) and to exploit 2MB regions only opportunistically, when sufficiently large contiguous regions are available.

Revelator+THP. Based on this observation, we build Revelator+THP, a variant of Revelator that combines opportunistic 2MB placement with the base 4KB tiered hash-based allocation. Revelator+THP does not rely on large contiguous regions being available. It first attempts to exploit 2MB large-page benefits when possible, and then falls back to Revelator’s 4KB hash-based placement when large pages are unavailable, ensuring robust performance across different fragmentation levels. On each data allocation, the OS proceeds as follows:

(1) Hash-based 2MB allocation. For a 2MB-aligned virtual region, the allocator first tries a hash-based 2MB page allocation. It hashes the 2MB-aligned virtual address to select a candidate 2MB-aligned physical region and checks whether the entire region is available. If so, the OS allocates that physical region, enabling the hardware to predict data placement at the 2MB granularity.

(2) Conventional THP fallback. If hash-based 2MB allocation fails, the OS falls back to the conventional Linux THP allocation scheme [93]. THP attempts to find and directly allocate any available free 2MB region. If successful, the page is allocated as a 2MB page, and the hardware can predict its placement at the 2MB granularity.

(3) Hash-based 4KB allocation. If no 2MB physical region is free, the allocator falls back to Revelator’s tiered hash-based 4KB allocation. To prevent interfering with future THP opportunities, 4KB hash-based allocation is strictly restricted to physical memory regions not already marked or reserved for 2MB pages.

(4) Buddy allocator fallback. As a last resort, the system falls back to the conventional buddy allocator to place the page in any arbitrary 4KB frame.

4.5. Revelator in Virtualized Environments

Revelator’s functionality naturally extends to virtualized systems. In this section, we describe Revelator in the context of x86-64 virtualization with Nested Paging [109], where address translation requires a two-dimensional page table walk from guest virtual addresses to guest physical addresses and then to host physical addresses. Our insight is to leverage tiered hash-based allocation and hardware speculation in two *complementary* and *composable* ways. Figure 8 illustrates the two approaches.

Diagonal speculation. First, Revelator can be used to predict the final host physical address (hPA) directly from the guest virtual page number (gVPN), overlapping the *entire* 2D PTW with data fetching (*diagonal speculation*, ①). In this approach, the hypervisor employs tiered hash-based allocation and allocates hPAs using the guest virtual address as the input to the hash function ($Hash(gVA) = hPA$). We provide more details on how the hypervisor can implement diagonal speculation without guest cooperation in the extended version of this paper [114].

Horizontal speculation. Second, Revelator can make the guest-physical (gPA) to host-physical (hPA) mapping predictable by having the hypervisor allocate hPAs using Revelator’s standard hash-based policy, with the gPA as the hash input (§4.1). This enables *horizontal speculation* during nested page table walks. Once the guest PTW identifies the gPA of a guest page-table page, the hardware hashes that gPA to predict the hPA where the page resides. It then speculatively fetches the cache line containing the next-level guest PTE before the nested translation completes ②.

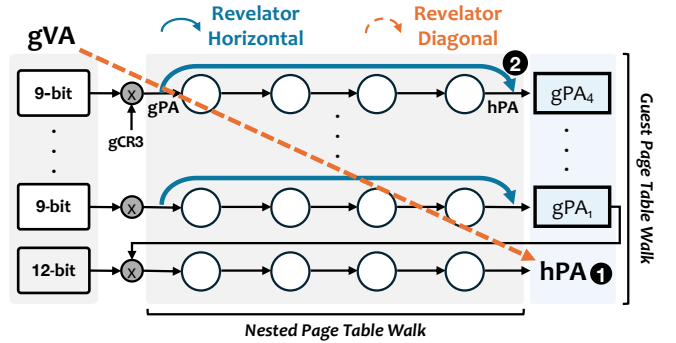


Figure 8: Revelator’s workflow in x86-64 virtualized environments that use Nested Paging [109].

4.6. Revelator in NUMA Systems

NUMA Placement and Speculation. In NUMA systems, the OS selects the node from which each page is allocated according to the active memory policy, such as Interleave, Local, Bind, or Preferred [122–124]. This node selection becomes part of the physical address, but for common policies such as Local, Bind, and Preferred, the hardware usually does not know the target node before address translation resolves. This uncertainty makes PA speculation challenging. Speculating across every node wastes bandwidth, while speculating only locally misses optimization opportunities for pages placed remotely due to policy, pressure, or migration. Existing contiguity-based mechanisms (e.g., SpOT [21]) face the same NUMA-local fragmentation problem, because they still depend on finding or preserving large contiguous regions within the selected node. If such schemes instead use the memory capacity of a remote node to preserve contiguity, the resulting access latency and interconnect traffic can offset their address translation benefits.

OS Side: Policy-First Node-Scoped Hashing. Revelator keeps NUMA placement under OS control because the selected NUMA node affects the latency and bandwidth of every later memory access to the page which is often more important than reducing address translation latency. During allocation, the OS first selects the target node n^* according to the active NUMA policy. Revelator then performs tiered hash-based

placement only within n^* 's address range, making intra-node 4KB placement predictable without changing the OS's locality decision. If all intra-node hash attempts fail, Revelator falls back to the conventional allocator within n^* . This policy-first design preserves OS locality decisions while avoiding the need to find large contiguous physical regions inside the selected node. We focus on Local, Bind, and Preferred, because unlike Interleave, their target node is not directly derivable from the VPN and is therefore challenging for PA speculation.

Hardware Side (1): Dominant-Driven Speculation. Figure 9 illustrates Revelator's NUMA-aware speculation policy. The speculation engine maintains per-NUMA-node counters C_n , incremented when a translation resolves and data resides in node n . These counters provide an empirical probability distribution, estimating the likelihood that data resides in node n ($p_n = C_n / \sum_i C_i$ ①). Let p_{dominant} be the probability of the *dominant* node with the highest counter value ($n_{\text{dom}} = \arg \max_n C_n$). (Note: the dominant node is not necessarily local). If $p_{\text{dominant}} \geq T_{\text{dom}}$ ($T_{\text{dom}} = 0.8$), Revelator speculates only within the dominant node.

Hardware Side (2): Hint-Assisted Speculation. If $p_{\text{dominant}} < T_{\text{dom}}$, no single node clearly dominates, increasing the chance that the page resides in a non-dominant node ②. Without identifying the target node, the engine must either speculate across every node (wasting bandwidth), or forfeit speculation. To avoid missing speculation opportunities, Revelator executes two concurrent actions: (i) speculates within the dominant node for likely hits, and (ii) performs a *hint walk*, a parallel lookup in a new OS-managed Bloom-filter table indexed by the VPN, to determine whether the page may reside in a non-dominant node ③. The hint walk is not part of the correctness-critical address translation path: it only returns a possible NUMA node for speculation, and the resolved page table translation still determines the correct physical address. If the hint points to a non-dominant node, Revelator issues one additional speculative fetch using the hash-derived candidate address within that node ④. We provide more details about this hint structure in the extended version of this work [114].

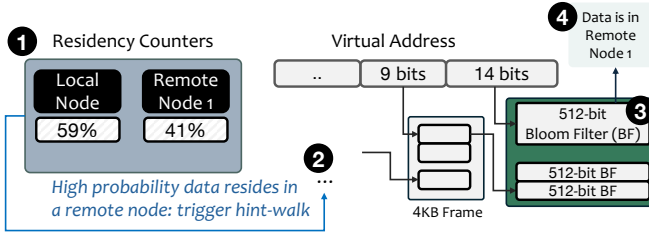


Figure 9: Revelator's workflow in NUMA systems.

5. System Integration

5.1. Security Considerations

Revelator *does not* introduce security risks beyond those of a typical cross-page hardware prefetcher [125–131]. First, Revelator does not allow the processor to use speculatively fetched data along a mis-speculated path, completely preventing Meltdown- or Spectre-style vulnerabilities [132, 133]. Second, Revelator limits the risk of cache side channels. An attacker could try to use a cache side-channel attack, such as Prime+Probe [134], to infer *when* a privileged physical address (PA_{secret}) is speculatively fetched. This does not reveal the

data stored at PA_{secret} , but it could leak coarse-grained information about system activity. Revelator mitigates this risk in two ways. First, it uses a secret, per-process hash key, making it difficult for an attacker to compute virtual to physical address collisions that could intentionally steer speculation toward a target address. Second, once the PTW resolves, hardware removes the microarchitectural trace of incorrect speculative fetches, narrowing the timing window available to a cache side-channel attack.

Speculative Fetch Tracking and Cleanup. For each page table walker, the speculation engine records the addresses of outstanding speculative fetches in a small bounded address log. The number of in-flight fetches is therefore limited by the number of page table walkers and the maximum speculation degree (N). When a speculatively fetched cache line arrives, Revelator installs it only in the private L2 cache, confining any transient microarchitectural footprint to the requesting core and preventing cross-core cache side channels.

When the PTW resolves, the MMU identifies the correct physical address and invalidates any incorrect speculative addresses recorded in the log. This cleanup uses standard invalidation logic and occurs off the critical path. Figure 10 quantifies why this cleanup significantly narrows the attack window. The left panel plots the time between speculative data arrival and PTW resolution: for 67% of mis-speculated fetches, the PTW resolves first, so the request can be cancelled in the MSHR [135] before it installs in L2. Only the remaining fetches arrive early enough to briefly enter the private L2. The right panel shows that, even for these installed lines, the exposure window is short: the average lifetime is 27 ns before invalidation. An attacker must therefore probe the victim's private L2 within this narrow interval, making reliable cache-timing attacks substantially harder than against conventional hardware prefetchers that leave longer-lived traces in shared cache structures.

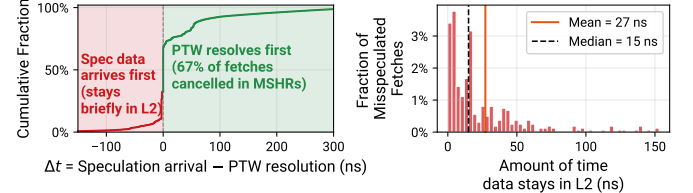


Figure 10: (Left) Timing of speculative fetch arrival relative to PTW resolution. (Right) Exposure window for cache blocks that arrive in the L2 cache before cleanup.

5.2. Integration into the Linux Kernel

We prototype Revelator in Linux 6.10.8 [95] and integrate the tiered hash-based allocation policy with the buddy allocator. Since Revelator probes at most a bounded number of hash-derived PPN candidates and performs constant-time metadata checks for each candidate, the hash-based allocation path performs $O(1)$ work per allocation. We provide extensive details about the Linux integration in the extended version of this paper [114].

Figure 11 shows the breakdown of Revelator's successful hash-based allocations across 6 allocation tiers, measured on a real system using our prototype implemented on top of Linux 6.10.8 [95]. During the experiments, a background memory hog continuously allocates and frees pages to keep memory utilization at $\sim 50\%$. To broaden workload diversity beyond the

workloads we evaluate in §7, we also evaluate database applications based on Cassandra [136], PyMongo [137], SQLite [138], and Spark [139], which are *not* bottlenecked by address translation. We make two observations. First, the first hash tier alone is sufficient to allocate on average 33–59% of pages, exceeding 50% for 8 of the 11 evaluated workloads. Second, using three tiers, Revelator successfully allocates on average 70–85% of pages in a hash-based manner, exceeding 80% for 7 of the 11 evaluated workloads.

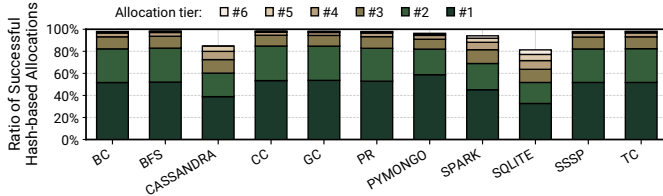


Figure 11: Breakdown of successful allocations using Revelator’s tiered hash-based allocation policy measured in a real Linux 6.10.8 prototype.

Minor Fault Latency. Figure 12 shows the fraction of total execution time spent in minor page faults for Revelator and baseline Linux across 10 workloads. We observe that even though baseline Linux’s minor fault latency is on average 19% lower than Revelator’s, minor page faults account for only a small fraction of total execution time in both systems (0.06% for Linux and 0.08% for Revelator on average). This behavior is expected for long-running translation-intensive workloads, which reach a steady state after their working set is fully mapped and produce no further page faults. Overall, Revelator’s additional fault-handling work has a negligible effect on end-to-end application execution time.

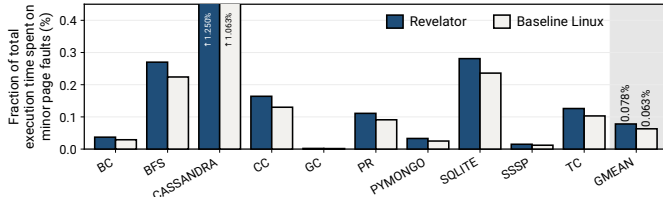


Figure 12: Fraction of total execution time spent on minor page faults in Revelator and baseline Linux across 10 different workloads.

5.3. Area and Power Overheads

We evaluate the area and power overheads introduced by Revelator’s hardware. We implement the speculation engine in Chisel [140] and synthesize it with Yosys [141] using the NanGate 45 nm open standard-cell library [142]. The synthesized engine occupies 0.0149 mm² of silicon area and dissipates 14.723 mW of static power. This corresponds to only 0.02% additional area and 0.03% additional power compared with a high-performance Intel Cascade Lake core [110].

6. Evaluation Methodology

We evaluate Revelator using Virtuoso [13, 96], a validated simulation methodology for accurately modeling the overheads of address translation, ported on top of Sniper [97, 98], an event-driven multicore simulator. We extend Virtuoso to model Revelator’s OS-level tiered hash-based allocation policy and Sniper to model Revelator’s hardware speculation engine. We use CityHash [115] as the hash function for both the OS and hardware components and charge a 2-cycle latency to produce the

hash result. Table 2 shows the details of the baseline simulated system configuration.

Table 2: Simulated Baseline System Configurations

Core	8-way OoO; ROB: 300 entries; x86-64 2.9 GHz
MMU	L1 I-TLB: 128-entry, 8-way assoc, 1-cycle
	L1 D-TLB (4 KB): 64-entry, 4-way assoc, 1-cycle; L1 D-TLB (2 MB): 32-entry, 4-way assoc, 1-cycle
	L2 TLB: 2048-entry, 16-way assoc, 12-cycle
	3 Page Structure Caches: 4,8,32-entry; 2-cycle
L1 Cache	64 KB, 8-way, 4-cycle; pLRU;
L2 Cache	1 MB, 16-way, 12-cycle; pLRU;
L3 Cache	2 MB/core, 16-way assoc, 35-cycle; SRRIP [143]
DRAM	128GB-DDR4-2400, 4 channels; 2 ranks/channel; 16 banks/rank; tCL = tRCD = tRP = 14.17 ns

Workloads. Table 3 lists all the workloads used in our evaluation. All evaluated workloads exhibit a high number of page table walks per kilo-instruction (PTWPKI) (>5), similar to prior studies [17, 19, 23, 144–146]. Each benchmark is executed for 300M instructions. In single-core systems, we evaluate the 11 most translation-intensive workloads from the GraphBench [99], XSBench [103], GUPS [100], DLRM [101], and GenomicsBench [102] suites. In multi-core and NUMA systems, we evaluate 30 mixes from 140 workloads from Google’s server suite [104, 147].

Evaluated Systems in Native Execution. We evaluate: (i) **THP**: Baseline system that utilizes Transparent Huge Pages (4KB/2MB). (ii) **ReserveTHP** [105]: reserves 2MB regions and promotes them based on the memory utilization of their resident 4KB pages, (iii) **SpecTLB** [88]: speculates physical addresses based on reserved large page regions, as described in §2, (iv) **POM-TLB** [107]: uses a 16MB software-managed L3 TLB, (v) **L2 TLB-64K**: uses a large 64K-entry L2 TLB, (vi) **ASAP** [108]: lays out page table data contiguously to enable page table prefetching, (vii) **DMT** [16]: establishes a direct mapping from virtual addresses to last-level PT entries, (viii) **ECH** [19]: employs the elastic cuckoo hash-based page table to improve parallelism in page table walks, (ix) **Mosaic-Pages** [106]: employs Iceberg hashing [148] to drastically increase the TLB reach, (x) **SpOT** [21]: a state-of-the-art speculative address translation scheme, described in detail in §2, (xi) **Revelator**: employs tiered hash-based allocation (§4.1) and speculation engine (§4.3) with 3 hashes and degree filtering (unless otherwise specified). (xii) **Revelator+THP**: Revelator combined with THP (§4.4). THP is enabled in L2 TLB-64K, SpOT, POM-TLB, ASAP, DMT, and ECH.

Memory Fragmentation & Utilization. We model fragmentation by controlling how many 2MB pages are available compared to the total possible 2MB pages in the system. We use three levels: high, medium, and low fragmentation, which correspond to 10%, 50%, and 90% of the total possible 2MB pages being available, respectively. These levels match measurements from our lab’s 22-node cluster (each node has 64 cores [149] and 256GB of memory), after approximately one month of uptime, one hour of uptime, and a fresh reboot, respectively. Fragmentation controls the available contiguity in free memory, whereas memory utilization controls how much memory is already occupied regardless of the data layout. We initialize each experiment with 20% base memory utilization by randomly marking 20% of the total physical pages as occupied.

Table 3: Evaluated Workloads

Suite	Workload	PTWPKI
GraphBIB [99]	Betweenness Centrality (BC), Breadth-first search (BFS), Connected components (CC), Graph coloring (GC), PageRank (PR), Triangle counting (TC), Single-source shortest path (SSSP)	10-45
	XSbench [103]	Particle Simulation (XS)
	GUPS [100]	Random-access (RND)
	DLRM [101]	Sparse-length sum (DLRM)
	GenomicsBench [102]	k-mer counting (GEN)
Google Workload Traces [104, 147]	140 traces from Sierra, Bravo, Tango, Tahoe, Merced, Yankee, Delta, Whiskey	5-22

7. Evaluation Results

7.1. Native Execution Environments

Fig. 13 shows the performance provided by Revelator and Revelator+THP compared to 8 state-of-the-art translation schemes (and a 64K-entry L2 TLB) across three fragmentation levels: (top) low, (middle) medium, and (bottom) high fragmentation. **Fast Page Table Walks.** In the low (medium) fragmentation scenario, Revelator outperforms ASAP, DMT, and ECH by 20% (13%), 19% (13%), and 20% (13%), respectively. While effective, these approaches parallelize or reduce the steps of the PTW but the processor still waits for at least one memory access before fetching the data. Revelator provides a complementary benefit by overlapping the final data fetch with the PTW, hiding the latency that remains even after the walk itself is accelerated.

Larger or Smarter TLBs. Under low fragmentation, L2 TLB-64K provides a small benefit over THP ($\approx 4\%$ on average), standalone Revelator roughly breaks even compared to the THP baseline system, and Revelator+THP improves performance by $\approx 12\%$ by exploiting abundant 2MB contiguity. This is expected: when 2MB contiguity is abundant, THP already provides high TLB reach while standalone Revelator does not filter 2MB translations in TLBs. As fragmentation increases, this trend reverses. Under medium fragmentation, Revelator improves performance by $\approx 19\%$ over THP, compared to only $\approx 2\%$ for L2 TLB-64K. Under high fragmentation, where most memory

is backed by 4KB pages, L2 TLB-64K improves performance by only $\approx 3\%$, while Revelator improves performance by $\approx 25\%$. Increasing L2 TLB capacity helps only when the additional entries convert misses into hits. This is more likely under low fragmentation, where 2MB pages are abundant and can be captured in TLBs, but becomes less likely as fragmentation increases and more translations require 4KB entries that are not present in the TLB. When a miss still occurs, the processor pays the full PTW cost. Revelator targets this residual cost by starting the data fetch during the PTW, so it can hide latency even for translations that a larger TLB fails to capture.

Speculation-based Schemes. We compare Revelator against SpOT [21], a state-of-the-art contiguity-based speculative address translation technique analyzed in §2. Under low fragmentation, SpOT improves performance by 11% over THP by exploiting large contiguous mappings, matching the performance of Revelator+THP. Standalone Revelator roughly breaks even compared to the THP baseline system in this setting because it does not leverage 2MB pages being translated by the TLBs, while SpOT and Revelator+THP do by design. Under medium fragmentation, SpOT’s speedup drops to 9% as contiguous mappings become harder to preserve. In contrast, Revelator improves performance by 19%, outperforming SpOT by 8.5%, because its hash-based placement does not rely on physical contiguity. Under high fragmentation, SpOT’s speedup drops further to 8%, while Revelator and Revelator+THP improve performance by 25% and 26%, respectively. This shows that Revelator+THP preserves the benefits of large pages when contiguity exists, while falling back to 4KB hash-based placement and speculation to maintain high performance when contiguity is scarce.

Understanding Revelator’s Performance. Fig. 14 breaks down how SpOT (without THP enabled), Revelator, and Revelator+THP affect address translation and data fetch latency in a system with medium fragmentation across 11 data-intensive workloads. SpOT and standalone Revelator respectively increase address translation latency over THP by 28% and 23% on average. THP remains a strong translation baseline at medium fragmentation: many translations are served by the L2 TLB, and PTWs terminate at the 3rd level due to the presence of 2MB

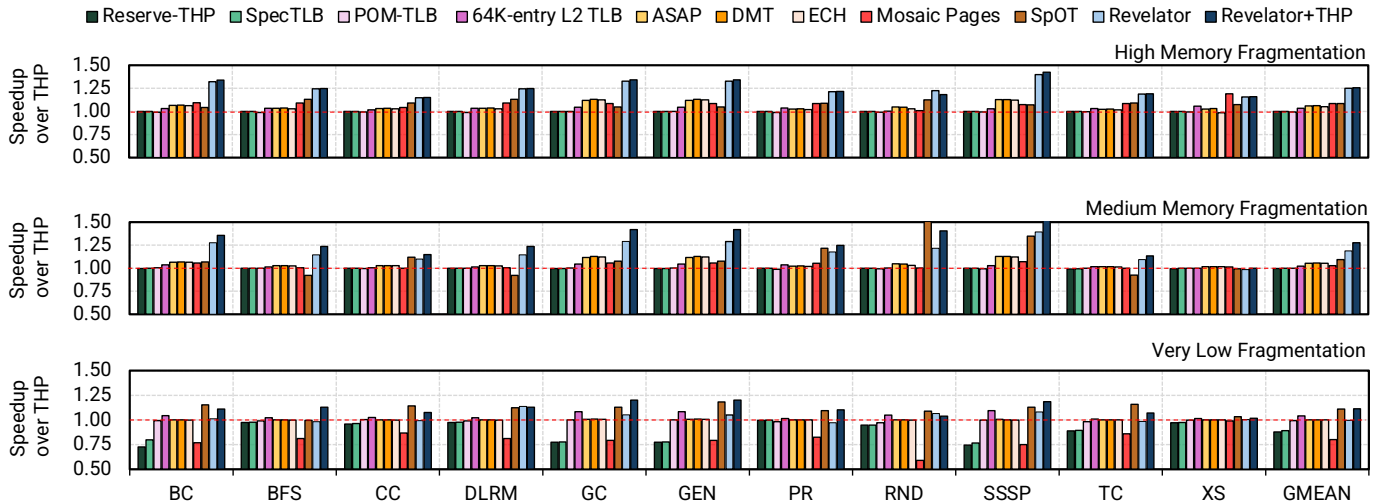


Figure 13: Performance speedup achieved by ReserveTHP [105], SpecTLB [88], L2 TLB-64K, POM-TLB [107], ASAP [108], DMT [16], ECH [19], Mosaic Pages [106], SpOT [21], Revelator and Revelator+THP over THP [105] across three fragmentation levels.

pages. Revelator+THP reduces address translation latency by 13% by preserving available 2MB mappings while accelerating PTWs for the remaining 4KB translations. All three schemes improve data fetch latency by issuing data requests before address translation completes: SpOT, Revelator, and Revelator+THP respectively reduce data fetching by 28%, 31%, and 33% on average.

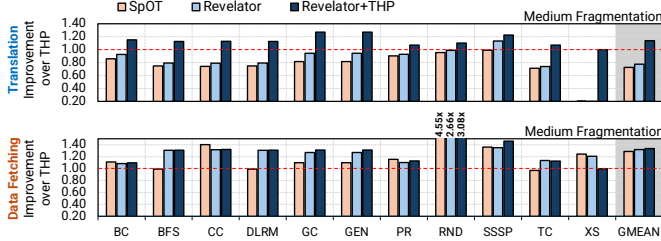


Figure 14: Improvement in address translation latency (top), and data fetch latency (bottom) achieved by SpOT [21], Revelator and Revelator+THP over THP (medium fragmentation) across 11 data-intensive workloads.

Impact of Hash Functions and Memory Utilization. Figure 15 shows Revelator’s speedup over THP with up to 5 allocation tiers (i.e., hash attempts) as memory utilization increases from 0% to 80% in a system with medium fragmentation. Revelator maintains an 8% average speedup even under high memory pressure, where 60% of pages cannot be allocated via a hash function. The best tier count depends on utilization: one tier performs best at 0% utilization because the primary target is usually available, while two tiers help at 40% utilization by increasing the chance of finding a free target. Using too many tiers adds redundant speculation overhead, causing a 4% slowdown with four tiers at 80% utilization. These results show that the useful speculation degree should track how many allocation tiers are likely to succeed at a given memory utilization: each additional tier creates another hash-derived candidate address that hardware may speculatively fetch, but fetching candidates from unlikely tiers wastes bandwidth. This motivates the speculation degree filter described in §4.3, which dynamically selects how many tier candidates Revelator should issue as speculative requests.

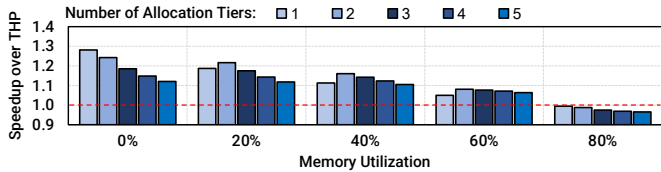


Figure 15: Average speedup over THP achieved by Revelator (medium fragmentation) with up to 5 allocation tiers (i.e., hash attempts) at different memory utilization levels across 11 data-intensive workloads.

Additional Sensitivity Studies. We further evaluate three sources of Revelator’s behavior: (1) the contribution of speculating for page table entries versus the final data fetch, (2) the speculation degree filter’s ability to reduce bandwidth overhead, and (3) the cache pollution caused by mis-speculation. These studies show that Revelator’s speedup primarily comes from hiding the final data fetch, that degree filtering reduces wasted fetches without sacrificing performance, and that cache blocks of mis-speculated data rarely evict useful data from the L2 cache. We provide the detailed analysis for all three studies in the extended version of this paper [114].

Impact on Energy Consumption. Using McPAT [150], in Fig. 16 we evaluate the energy consumption of Mosaic Pages, SpOT, Revelator, and Revelator+THP compared to THP under medium fragmentation across 11 data-intensive workloads. We observe that Revelator+THP reduces energy consumption by 5.5% over THP and 2.2%, 4.3%, and 2.7% over Mosaic Pages, SpOT, and standalone Revelator, respectively. We provide a comprehensive energy analysis in the extended version of the paper [114].

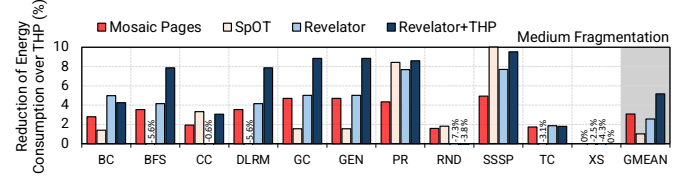


Figure 16: Reduction in energy consumption achieved by Mosaic Pages [106], SpOT [21], Revelator and Revelator+THP over THP in a system with medium fragmentation across 11 data-intensive workloads.

Revelator in Virtualized Environments. We evaluate Revelator in virtualized environments and compare Nested Paging+THP (NP-THP) against DMT [16] and three Revelator configurations: Horizontal (§4.5), Diagonal (§4.5) and Full (§4.5), which combines both forms of speculation. Horizontal provides little benefit because the nested translations are often retrieved by the nested TLB, while Diagonal improves performance by 11% over NP-THP and 6% over DMT. Full combines both forms of speculation and reaches 13.6% average speedup over NP-THP. We provide extended analysis on virtualized environments in the extended version of the paper [114].

Revelator in Multicore Systems. We evaluate Revelator and THP in 4-, 8- and 16-core configurations across 30 mixes of Google server traces (we compare against SpOT only in the multicore-NUMA evaluation to reduce the total volume of experiments). Fig. 17 (left) shows that Revelator consistently outperforms THP as core count increases, providing average speedups ranging from $1.08\times$ under low fragmentation on 4 cores to $1.50\times$ under high fragmentation on 16 cores. The benefit grows with core count because more applications compete for 2MB pages, causing THP to fall back to 4KB pages more often while Revelator does not rely on contiguity and continues to hide translation overhead. Fig. 17 (right) supports this trend: THP’s L2 TLB MPKI increases by more than 20% from 4 to 16 cores, while Revelator’s speculation accuracy remains stable at 87-88%.

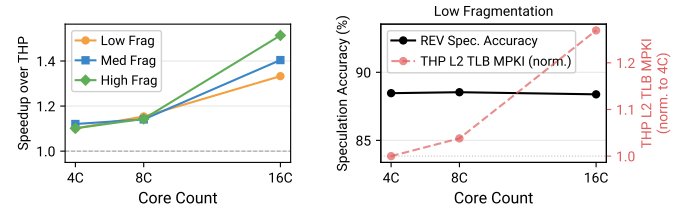


Figure 17: (Left) Average speedup achieved by Revelator over THP under low/medium/high fragmentation; (right) L2 TLB MPKI increase for THP versus speculation accuracy for Revelator under low fragmentation across 4/8/16-core systems.

Revelator in NUMA Systems. We evaluate Revelator in an 8-node/64-core NUMA system across 30 mixes of Google server traces while varying the spillover rate from 0% to 50%, where spillover is the fraction of an application’s pages allocated out-

side its dominant NUMA node. The OS policy allocates pages on the dominant node (preferred) until it reaches a per-node memory utilization threshold, at which point it starts allocating on other nodes, increasing spillover. Fig. 18 shows Revelator’s performance speedup over SpOT [21] and the fraction of PTW latency hidden by Revelator (Hints ON/OFF §4.6) and SpOT. Revelator consistently outperforms SpOT, retaining an average $1.08\times$ speedup even at 50% spillover (up to $1.20\times$ speedup at 0% spillover), where many pages reside outside the dominant node. Bloom-filter-based hints sustain this benefit by directing speculation to likely remote nodes, allowing Revelator to hide 85–95% of PTW latency across spillover rates. We provide a detailed NUMA analysis, including additional system configurations and bandwidth overheads, in the extended version of the paper [114].

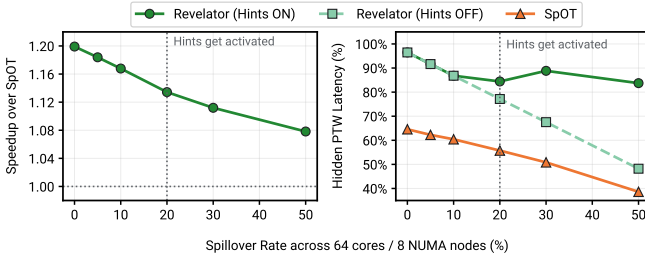


Figure 18: Average speedup and hidden PTW latency achieved by Revelator with Hints ON/OFF over SpOT in an 8-node/64-core NUMA system at different spillover rates across 30 mixes of Google server traces.

8. Related Work

To our knowledge, Revelator is the first work that employs hashing to enable highly accurate speculative address translation. §7.1 quantitatively compares Revelator with 8 state-of-the-art mechanisms that mitigate translation overheads via faster page table walks [16, 19, 108], larger TLB reach [105–107] and contiguity-based speculative address translation [21, 88]. This section places Revelator in the context of prior work that leverages hashing, large pages, contiguity, and TLB optimizations to mitigate address translation overheads.

Hash-based Address Translation & Page Tables. Several works [9, 17, 19, 106, 116, 117, 146, 151, 152] leverage hashing to make translation structures smaller, faster, or increase parallelism. EMT [153] provides an OS framework for designing such new address translation architectures, which could facilitate Revelator’s integration in future systems. Revelator can be tailored to work in tandem with schemes [17, 106] that employ hash-based virtual-to-physical mappings to perform fast address translation (see extended version [114]), and is fully orthogonal to hash-based page tables: Revelator hides the latency of PTWs while hash-based page tables make PTWs faster.

Using Large Pages. A large body of works leverages large pages to mitigate address translation overheads [90, 118, 121, 154–166]. However, they require contiguous physical memory, which can be scarce in fragmented systems [5, 91, 92]. Prior work improves large-page availability despite fragmentation, by allowing holes inside 2MB pages [154], by supporting more flexible page sizes [118, 119] or by selectively allocating large pages based on runtime conditions [155, 158, 159]. In §4.4 and §7.1, we show that Revelator works synergistically with large pages. Revelator+THP preserves the benefits of 2MB pages

when they are available, while Revelator continues to accelerate address translation for 4KB pages when fragmentation prevents large-page allocation.

Leveraging Contiguity. Another class of works [1, 5, 16, 20, 21, 108, 120, 145, 167–169] proposes increasing the effective address translation reach by exploiting contiguous virtual-to-physical mappings. PTE coalescing mechanisms such as CoLT [169, 170] opportunistically merge adjacent contiguous translations into larger TLB entries, while Hybrid TLB Coalescing [120] encodes contiguity in PTEs to adapt coalescing granularity under fragmented allocations. These mechanisms are effective when contiguous ranges can be formed, but their benefits diminish as physical memory becomes fragmented. As we show in §7.1, Revelator’s hash-based placement enables hardware to predict data and PTE locations without depending on the availability of contiguous physical ranges, allowing Revelator to remain effective under high fragmentation.

Large TLBs, TLB Management & TLB Prefetching. A broad class of works reduces the frequency of PTWs by increasing the TLB reach or by fetching translations before they are demanded [3, 18, 108, 171–187]. Large or smarter TLB organizations [172–179, 188], additional TLB levels [186, 187, 189], and software-managed TLBs [107, 184, 190–195] increase the number of translations that can be cached on-chip. TLB prefetchers predict future translations and insert them ahead of use [180–184], while replacement policies [171, 196] and cache-backed TLB storage [18, 197, 198] keep useful translations resident for longer. These mechanisms are orthogonal to Revelator: they reduce how often PTWs occur, whereas Revelator hides the latency of PTWs that TLBs cannot eliminate.

9. Conclusion

This paper introduces Revelator, a hardware-OS cooperative mechanism that leverages hash-based memory allocation to enable highly accurate, low-cost speculative address translation. Revelator uses tiered hash-based allocation for both program data and last-level PTEs, enabling lightweight hardware to predict where the data and the final PTE are likely to reside after an L2 TLB miss. By issuing these requests before conventional translation completes, Revelator both hides the address translation latency and accelerates PTWs. Across single-core, multi-core, virtualized, and NUMA systems, we show that Revelator delivers substantial performance and energy benefits with very small hardware and OS modifications. Beyond conventional CPU-based systems, we hope that Revelator’s OS-guided predictability provides a foundation for improving address translation in heterogeneous platforms that combine CPUs, GPUs, accelerators and other emerging memory technologies and computing paradigms. Revelator is freely available at <https://github.com/CMU-SAFARI/Virtuoso>.

10. Acknowledgments

We thank the anonymous reviewers of MICRO 2025, ASPLOS 2026 and ISCA 2026 for their feedback. We thank the SAFARI Research Group members for their constructive feedback and for providing a stimulating intellectual and scholarly environment. We acknowledge the generous gift funding provided by our industrial partners (especially Google, Huawei, Intel, Microsoft), which has been instrumental in enabling the research we have been conducting on memory systems.

References

- [1] Arkaprava Basu, Jayneel Gandhi, Jichuan Chang, Mark D. Hill, and Michael M. Swift. Efficient Virtual Memory for Big Memory Servers. In *ISCA*, 2013.
- [2] Vasileios Karakostas, Osman S. Unsal, Mario Nemirovsky, Adrian Cristal, and Michael Swift. Performance Analysis of the Memory Management Unit Under Scale-Out Workloads. In *IISWC*, 2014.
- [3] Thomas W. Barr, Alan L. Cox, and Scott Rixner. Translation Caching: Skip, Don’t Walk (the Page Table). In *ISCA*, 2010.
- [4] Linux. 5 Level Paging. <https://docs.kernel.org/x86/x8664/5level-paging.html>, 2021.
- [5] Kaiyang Zhao, Kaiwen Xue, Ziqi Wang, Dan Schatzberg, Leon Yang, Antonis Manousis, Johannes Weiner, Rik Van Riel, Bikash Sharma, Chunqiang Tang, and Dimitrios Skarlatos. Contiguitas: the Pursuit of Physical Memory Contiguity in Datacenters. In *ISCA*, 2023.
- [6] Sandeep Kumar, Aravinda Prasad, Smruti R. Sarangi, and Sreenivas Subramoney. Radiant: Efficient Page Table Management for Tiered Memory Systems. In *ISMM*, 2021.
- [7] Abhishek Bhattacharjee and Margaret Martonosi. Characterizing the TLB Behavior of Emerging Parallel Workloads On Chip Multiprocessors. In *PACT*, 2009.
- [8] Swapnil Haria, Mark D. Hill, and Michael M. Swift. De-virtualizing Memory in Heterogeneous Systems. In *ASPLOS*, 2018.
- [9] Idan Yaniv and Dan Tsafir. Hash, Don’t Cache (the Page Table). In *SIGMETRICS*, 2016.
- [10] Timothy Merrifield and H. Reza Taheri. Performance Implications of Extended Page Tables On Virtualized X86 Processors. In *VEE*, 2016.
- [11] Peter Hornyack, Luis Ceze, Steve Gribble, Dan Ports, and Hank Levy. A Study of Virtual Memory Usage and Implications for Large Memory. Technical report, University of Washington, 2013.
- [12] Nick Lindsay and Abhishek Bhattacharjee. Understanding Address Translation Scaling Behaviours Using Hardware Performance Counters. In *IISWC*, 2024.
- [13] Konstantinos Kanellopoulos, Konstantinos Sgouras, F. Nisa Bostanci, Andreas Kosmas Kakolyris, Berkin Kerim Konar, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Nandita Vijaykumar, and Onur Mutlu. Virtuoso: Enabling Fast and Accurate Virtual Memory Research via an Imitation-based Operating System Simulation Methodology. In *ASPLOS*, 2025.
- [14] Konstantinos Kanellopoulos. *Cutting Through the Virtual Memory Wall: From Rigid Abstractions to High-Performance, Flexible, Efficient Cooperation Between Hardware and System Software*. PhD thesis, ETH Zurich, 2026.
- [15] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. Profiling a Warehouse-scale Computer. In *ISCA*, 2015.
- [16] Jiyuan Zhang, Weiwei Jia, Siyuan Chai, Peizhe Liu, Jongyul Kim, and Tianyin Xu. Direct Memory Translation for Virtualized Clouds. In *ASPLOS*, 2024.
- [17] Konstantinos Kanellopoulos, Rahul Bera, Kosta Stojiljkovic, Nisa Bostanci, Can Firtina, Rachata Ausavarungnirun, Rakesh Kumar, Nastaran Hajinazar, Jisung Park, Mohammad Sadrosadati, Nandita Vijaykumar, and Onur Mutlu. Utopia: Efficient Address Translation using Hybrid Virtual-to-Physical Address Mapping. In *MICRO*, 2023.
- [18] Konstantinos Kanellopoulos, Hong Chul Nam, F. Nisa Bostanci, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Davide Basilio Bartolini, and Onur Mutlu. Vic-tima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources. In *MICRO*, 2023.
- [19] Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, and Josep Torrellas. Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism. In *ASPLOS*, 2020.
- [20] Vasileios Karakostas, Jayneel Gandhi, Furkan Ayar, Adrián Cristal, Mark D. Hill, Kathryn S. McKinley, Mario Nemirovsky, Michael M. Swift, and Osman Ünsal. Redundant Memory Mappings for Fast Access to Large Memories. In *ISCA*, 2015.
- [21] Chloe Alverti, Stratos Psomadakis, Vasileios Karakostas, Jayneel Gandhi, Konstantinos Nikas, Georgios Goumas, and Nectarios Koziris. Enhancing and Exploiting Contiguity for Fast Memory Virtualization. In *ISCA*, 2020.
- [22] Nastaran Hajinazar, Pratyush Patel, Minesh Patel, Konstantinos Kanellopoulos, Saugata Ghose, Rachata Ausavarungnirun, Geraldo F. Oliveira, Jonathan Appavoo, Vivek Seshadri, and Onur Mutlu. The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework. In *ISCA*, 2020.
- [23] Siddharth Gupta, Atri Bhattacharyya, Yunho Oh, Abhishek Bhattacharjee, Babak Falsafi, and Mathias Payer. Rebooting Virtual Memory with Midgard. In *ISCA*, 2021.
- [24] Irina Calciu, Ivan Puddu, Aasheesh Kolli, Andreas Nowatzky, Jayneel Gandhi, Onur Mutlu, and Pratap Subrahmanyam. Project pberry: Fpga acceleration for remote memory. In *HotOS*, 2019.
- [25] Peter X. Gao, Akshay Narayan, Sagar Karandikar, Joao Carreira, Sangjin Han, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. Network Requirements for Resource Disaggregation. In *OSDI*, 2016.
- [26] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *OSDI*, 2018.
- [27] Dario Korolija, Dimitrios Koutsoukos, Kimberly Keeton, Konstantin Taranov, Dejan S. Milojevic, and Gustavo Alonso. Farview: Disaggregated Memory with Operator Off-loading for Database Engines. In *CIDR*, 2022.
- [28] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D. Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. Semeru: A Memory-Disaggregated Managed Runtime. In *OSDI*, 2020.
- [29] Pengfei Zuo, Jiazhao Sun, Liu Yang, Shuangwu Zhang, and Yu Hua. One-sided RDMA-Conscious Extendible Hashing for Disaggregated Memory. In *ATC*, 2021.

- [30] Hasan Al Maruf and Mosharaf Chowdhury. Effectively Prefetching Remote Memory with Leap. In *ATC*, 2020.
- [31] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K. Reinhardt, and Thomas F. Wenisch. Disaggregated Memory for Expansion and Sharing in Blade Servers. In *ISCA*, 2009.
- [32] Qizhen Zhang, Yifan Cai, Sebastian Angel, Vincent Liu, Ang Chen, and Boon Thau Loo. Rethinking Data Management Systems for Disaggregated Data Centers. In *CIDR*, 2020.
- [33] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhat-tacharjee. Nimble Page Management for Tiered Memory Systems. In *ASPLOS*, 2019.
- [34] Sebastian Angel, Mihir Nanavati, and Siddhartha Sen. Disaggregation and the Application. In *HotCloud*, 2020.
- [35] Kevin Lim, Yoshio Turner, Jose Renato Santos, Alvin AuYoung, Jichuan Chang, Parthasarathy Ranganathan, and Thomas F. Wenisch. System-Level Implications of Disaggregated Memory. In *HPCA*, 2012.
- [36] Ivy Peng, Roger Pearce, and Maya Gokhale. On the Memory Underutilization: Exploring Disaggregated Memory on HPC Systems. In *SBAC-PAD*, 2020.
- [37] Laurent Bindschaedler, Ashvin Goel, and Willy Zwaenepoel. Hailstorm: Disaggregated Compute and Storage for Distributed LSM-Based Databases. In *ASPLOS*, 2020.
- [38] K. Katrinis, D. Syrivelis, D. Pnevmatikatos, G. Zervas, D. Theodoropoulos, I. Koutsopoulos, K. Hasharoni, D. Raho, C. Pinto, F. Espina, S. Lopez-Buedo, Q. Chen, M. Nemirovsky, D. Roca, H. Klos, and T. Berends. Rack-Scale Disaggregated Cloud Data Centers: The dReDBox Project Vision. In *DATE*, 2016.
- [39] Marcos K. Aguilera, Nadav Amit, Irina Calciu, Xavier Deguillard, Jayneel Gandhi, Pratap Subrahmanyam, Lalith Suresh, Kiran Tati, Rajesh Venkatasubramanian, and Michael Wei. Remote Memory in the Age of Fast Networks. In *SoCC*, 2017.
- [40] Marcos K. Aguilera, Nadav Amit, Irina Calciu, Xavier Deguillard, Jayneel Gandhi, Stanko Novakovic, Arun Ramanathan, Pratap Subrahmanyam, Lalith Suresh, Kiran Tati, Rajesh Venkatasubramanian, and Michael Wei. Remote Regions: A Simple Abstraction for Remote Memory. In *ATC*, 2018.
- [41] Pramod Subba Rao and George Porter. Is Memory Disaggregation Feasible? A Case Study with Spark SQL. In *ANCS*, 2016.
- [42] Irina Calciu, M. Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. Rethinking Software Runtimes for Disaggregated Memory. In *ASPLOS*, 2021.
- [43] Atul Adya, Robert Grandl, Daniel Myers, and Henry Qin. Fast Key-Value Stores: An Idea Whose Time Has Come and Gone. In *HotOS*, 2019.
- [44] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, Greg Thelen, Kamil Adam Yurtsever, Yu Zhao, and Parthasarathy Ranganathan. Software-Defined Far Memory in Warehouse-Scale Computers. In *ASPLOS*, 2019.
- [45] Christian Pinto, Dimitris Syrivelis, Michele Gazzetti, Panos Koutsovasilis, Andrea Reale, Kostas Katrinis, and H. Peter Hofstee. ThymesisFlow: A Software-Defined, HW/SW co-Designed Interconnect Stack for Rack-Scale Memory Disaggregation. In *MICRO*, 2020.
- [46] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. Efficient Memory Disaggregation with Infiniswap. In *NSDI*, 2017.
- [47] Dhantu Buragohain, Abhishek Ghogare, Trishal Patel, Mythili Vutukuru, and Purushottam Kulkarni. DiME: A Performance Emulator for Disaggregated Memory Architectures. In *APSys*, 2017.
- [48] Georgios Zervas, Hui Yuan, Arsalan Saljoghei, Qianqiao Chen, and Vaibhawa Mishra. Optically Disaggregated Data Centers with Minimal Remote Memory Latency: Technologies, Architectures, and Resource Allocation. In *JOCN*, 2018.
- [49] HanBin Yoon, Justin Meza, Rachata Ausavarungnirun, Rachael A Harding, and Onur Mutlu. Row Buffer Locality Aware Caching Policies for Hybrid Memories. In *ICCD*, 2012.
- [50] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, Onur Mutlu, and Srinivas Devadas. Banshee: Bandwidth-Efficient DRAM Caching Via Software/Hardware Cooperation. In *MICRO*, 2017.
- [51] Yang Li, Saugata Ghose, Jongmoo Choi, Jin Sun, Hui Wang, and Onur Mutlu. Utility-Based Hybrid Memory Management. In *CLUSTER*, 2017.
- [52] Jishen Zhao, Onur Mutlu, and Yuan Xie. FIRM: Fair and High-Performance Memory Control for Persistent Memory Systems. In *MICRO*, 2014.
- [53] Reza Salkhordeh, Onur Mutlu, and Hossein Asadi. An Analytical Model for Performance and Lifetime Estimation of Hybrid DRAM-NVM Main Memories. In *TC*, 2019.
- [54] Justin Meza, Jichuan Chang, HanBin Yoon, Onur Mutlu, and Parthasarathy Ranganathan. Enabling Efficient and Scalable Hybrid Memories using Fine-granularity DRAM Cache Management. In *CAL*, 2012.
- [55] Luiz Ramos, Eugene Gorbato, and Ricardo Bianchini. Page Placement in Hybrid Memory Systems. In *ICS*, 2011.
- [56] Chenxi Wang, Huimin Cui, Ting Cao, John N. Zigman, Haris Volos, Onur Mutlu, Fang Lv, Xiaobing Feng, and Guoqing Harry Xu. Panthera: Holistic Memory Management for Big Data Processing over Hybrid Memories. In *PLDI*.
- [57] Lei Chen, Jiacheng Zhao, Chenxi Wang, Ting Cao, John N. Zigman, Haris Volos, Onur Mutlu, Fang Lv, Xiaobing Feng, Guoqing Harry Xu, and Huimin Cui. Unified Holistic Memory Management Supporting Multiple Big Data Processing Frameworks over Hybrid Memories. *ACM Trans. Comput. Syst.*, 2021.
- [58] Geraldo F. Oliveira, Saugata Ghose, Juan Gómez-Luna, Amirali Boroumand, Alexis Savery, Sonny Rao, Salman Qazi, Gwendal Grignou, Rahul Thakur, Eric Shiu, and Onur Mutlu. Extending Memory Capacity in Modern Consumer Systems With Emerging Non-Volatile Mem-

- ory: Experimental Analysis and Characterization Using the Intel Optane SSD. *IEEE Access*, 2023.
- [59] Rakesh Nadig, Vamanan Arulchelvan, Rahul Bera, Taha Shahroodi, Gagandeep Singh, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. Harmonia: A Multi-Agent Reinforcement Learning Approach to Data Placement and Migration in Hybrid Storage Systems. *CoRR*, abs/2503.20507, 2025.
- [60] Shihao Song, Anup Das, Onur Mutlu, and Nagarajan Kandasamy. Improving Phase Change Memory Performance with Data Content Aware Access. In *ISMM*, 2020.
- [61] Apostolos Kokolis, Dimitrios Skarlatos, and Josep Torrellas. PageSeer: Using Page Walks to Trigger Page Swaps in Hybrid Memory Systems. In *HPCA*, pages 596–608, 2019.
- [62] Matthew Benjamin Olson, Brandon Kammerdiener, Michael R. Jantz, Kshitij A. Doshi, and Terry R. Jones. Online Application Guidance for Heterogeneous Memory Systems. *ACM Trans. Archit. Code Optim.*, 19(3):45:1–45:27, 2022.
- [63] Fei Wen, Mian Qin, Paul V. Gratz, and A. L. Narasimha Reddy. Hardware Memory Management for Future Mobile Hybrid Memory Systems. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 39(11):3627–3637, 2020.
- [64] Sihang Liu, Korakit Seemakhupt, Gennady Pekhimenko, Aasheesh Kolli, and Samira Khan. Janus: Optimizing Memory and Storage Support for Non-Volatile Memory Systems. In *ISCA*, 2019.
- [65] Chloe Alverti, Vasileios Karakostas, Nikhita Kunati, Georgios Goumas, and Michael Swift. DaxVM: Stressing the Limits of Memory as a File Interface. In *MICRO*, 2022.
- [66] Pirmin Vogel, Andrea Marongiu, and Luca Benini. Lightweight Virtual Memory Support for Zero-Copy Sharing of Pointer-Rich Data Structures in Heterogeneous Embedded SoCs. *IEEE Trans. Parallel Distributed Syst.*, 28(7):1947–1959, 2017.
- [67] Pirmin Vogel, Andrea Marongiu, and Luca Benini. Exploring Shared Virtual Memory for FPGA Accelerators with a Configurable IOMMU. *IEEE Trans. Computers*, 68(4):510–525, 2019.
- [68] Pirmin Vogel. *Shared Virtual Memory for Heterogeneous Embedded Systems on Chips*. PhD thesis, ETH Zurich, 2018.
- [69] Andreas Kurth, Pirmin Vogel, Andrea Marongiu, and Luca Benini. Scalable and Efficient Virtual Memory Sharing in Heterogeneous SoCs with TLB Prefetching and MMU-Aware DMA Engine. In *ICCD*, 2018.
- [70] Canberk Sönmez, Mohamed Shahawy, Cemalettin Cem Belentepe, and Paolo Ienne. FlexiMem: Flexible Shared Virtual Memory for PCIe-attached FPGAs. In *FPL*, 2024.
- [71] Edward Richter and Deming Chen. Qilin: Enabling Performance Analysis and Optimization of Shared-Virtual Memory Systems with FPGA Accelerators. In *ICCAD*, 2022.
- [72] Bennett Cooper, Thomas R. W. Scogland, and Rong Ge. Shared Virtual Memory: Its Design and Performance Implications for Diverse Applications. In *ICS*, 2024.
- [73] Torben Kalkhof and Andreas Koch. Efficient Physical Page Migrations in Shared Virtual Memory Reconfigurable Computing Systems. In *FPT*, 2021.
- [74] Jihun Park, Donghun Jeong, and Jungrae Kim. UVMMU: Hardware-Offloaded Page Migration for Heterogeneous Computing. In *DATE*, 2023.
- [75] Duheon Choi, Taeyang Jeong, Joonhyeok Yeom, and Eui-Young Chung. Operand-Oriented Virtual Memory Support for Near-Memory Processing. *IEEE Trans. Computers*, 72(8):2250–2263, 2023.
- [76] Yueqi Wang, Bingyao Li, Mohamed Tarek Ibn Ziad, Lieven Eeckhout, Jun Yang, Aamer Jaleel, and Xulong Tang. OASIS: Object-Aware Page Management for Multi-GPU Systems. In *HPCA*, 2025.
- [77] Yueqi Wang, Bingyao Li, Aamer Jaleel, Jun Yang, and Xulong Tang. GRIT: Enhancing Multi-GPU Performance with Fine-Grained Dynamic Page Placement. In *HPCA*, 2024.
- [78] Pratheek B, Guilherme Cox, Jan Vesely, and Arkaprava Basu. SUV: Static Analysis Guided Unified Virtual Memory. In *MICRO*, 2024.
- [79] Hyunkyun Shin, Seongtae Bang, Hyungwon Park, and Daehoon Kim. ARIADNE: Adaptive UVM Management for Efficient GPU Memory Oversubscription. In *HPCA*, 2026.
- [80] Junhyeok Park, Sungbin Jang, Osang Kwon, Yongho Lee, and Seokin Hong. Leveraging Chiplet-Locality for Efficient Memory Mapping in Multi-Chip Module GPUs. In *MICRO*, 2025.
- [81] Yeonan Ha, Jiho Park, Hanna Cha, Jiwon Lee, Joonsung Kim, Won Woo Ro, and Youngsok Kim. LATPC: Accelerating GPU Address Translation Using Locality-Aware TLB Prefetching and MSHR Compression. In *MICRO*, 2025.
- [82] Yuan Feng, Yuke Li, Jiwon Lee, Won Woo Ro, and Hyeran Jeon. Heliostat: Harnessing Ray Tracing Accelerators for Page Table Walks. In *ISCA*, 2025.
- [83] Sungbin Jang, Junhyeok Park, Yongho Lee, Osang Kwon, Donghyun Kim, Juyoung Seok, and Seokin Hong. SoftWalker: Supporting Software Page Table Walk for Irregular GPU Applications. In *MICRO*, 2025.
- [84] Sungbin Jang, Junhyeok Park, Osang Kwon, Yongho Lee, and Seokin Hong. Rethinking Page Table Structure for Fast Address Translation in GPUs: A Fixed-Size Hashed Page Table. In *PACT*, 2024.
- [85] Yuan Feng, Seonjin Na, Hyesoon Kim, and Hyeran Jeon. Barre Chord: Efficient Virtual Memory Translation for Multi-Chip-Module GPUs. In *ISCA*, 2024.
- [86] Bingyao Li, Yueqi Wang, Tianyu Wang, Lieven Eeckhout, Jun Yang, Aamer Jaleel, and Xulong Tang. STAR: Sub-Entry Sharing-Aware TLB for Multi-Instance GPU. In *MICRO*, pages 309–323, 2024.
- [87] Junsu Kim, Jaebeom Jeon, Jaeyong Park, Sangun Choi, Minseong Gil, Seokin Hong, Gunjae Koo, Myung Kuk Yoon, and Yunho Oh. MOST: Memory Oversubscription-Aware Scheduling for Tensor Migration on GPU Unified Storage. *CAL*, 2025.
- [88] Thomas W. Barr, Alan L. Cox, and Scott Rixner. SpecTLB: A Mechanism for Speculative Address Translation. In *ISCA*, 2011.

- [89] Junhyeok Park, Osang Kwon, Yongho Lee, Seongwook Kim, Gwangeun Byeon, Jihun Yoon, Prashant J. Nair, and Seokin Hong. A case for speculative address translation with rapid validation for gpus. In *MICRO*, 2024.
- [90] Binh Pham, Ján Veselý, Gabriel H. Loh, and Abhishek Bhattacharjee. Large Pages and Lightweight Memory Management in Virtualized Environments: Can You Have It Both Ways? In *MICRO*, 2015.
- [91] Mark Mansi, Bijan Tabatabai, and Michael M. Swift. CBMM: Financial Advice for Kernel Memory Managers. In *ATC*, 2022.
- [92] Mark Mansi and Michael M. Swift. Characterizing Physical Memory Fragmentation. In *arXiv*, 2024.
- [93] Jonathan Corbet. Transparent Huge Pages in 2.6.38. <https://lwn.net/Articles/423584/>, 2011.
- [94] Jonathan Corbet. The Current State of Kernel Page-Table Isolation. <https://lwn.net/Articles/741878/>, 2017.
- [95] The linux kernel 6.10.8. <https://lwn.net/Articles/988748/>.
- [96] SAFARI Research Group. Virtuoso. <https://github.com/CMU-SAFARI/Virtuoso>.
- [97] Trevor E. Carlson, Wim Heirman, and Lieven Eeckhout. Sniper: Exploring the Level of Abstraction for Scalable and Accurate Parallel Multi-Core Simulations. In *SC*, 2011.
- [98] Sniper. <https://github.com/snipersim/snipersim>.
- [99] Lifeng Nai, Yinglong Xia, Ilie G. Tanase, Hyesoon Kim, and Ching-Yung Lin. GraphBIG: Understanding Graph Computing in the Context of Industrial Solutions. In *SC*, 2015.
- [100] Steven J. Plimpton, Ron Brightwell, Courtenay Vaughan, Keith Underwood, and Mike Davis. A Simple Synchronous Distributed-Memory Algorithm for the HPC RandomAccess Benchmark. In *Cluster*, 2006.
- [101] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep Learning Recommendation Model for Personalization and Recommendation Systems, 2019.
- [102] Arun Subramaniyan, Yufeng Gu, Timothy Dunn, Somnath Paul, Md. Vasimuddin, Sanchit Misra, David Blaauw, Satish Narayanasamy, and Reetuparna Das. GenomicsBench: A Benchmark Suite for Genomics. In *ISPASS*, 2021.
- [103] John R. Tramm, Andrew R. Siegel, Tanzima Islam, and Martin Schulz. XSBench - The Development and Verification of a Performance Abstraction for Monte Carlo Reactor Analysis. In *PHYSOR*, 2014.
- [104] Google workload traces version 2. <https://console.cloud.google.com/storage/browser/external-traces-v2>. Accessed: 2026-03-06.
- [105] Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. Practical, Transparent Operating System Support for Superpages. In *OSDI*, 2002.
- [106] Krishnan Gosakan, Jaehyun Han, William Kuszmaul, Ibrahim Nael Mubarek, Nirjhar Mukherjee, Guido Tagliavini, Evan West, Michael Bender, Abhishek Bhattacharjee, Alex Conway, Martin Farach-Colton, Jayneel Gandhi, Rob Johnson, Sudarsun Kannan, and Donald Porter. Mosaic Pages: Big TLB Reach with Small Pages. In *ASPLOS*, 2023.
- [107] Jee Ho Ryoo, Nagendra Gulur, Shuang Song, and Lizy K. John. Rethinking TLB Designs in Virtualized Environments: A Very Large Part-of-Memory TLB. In *ISCA*, 2017.
- [108] Artemiy Margaritov, Dmitrii Ustiugov, Edouard Bugnion, and Boris Grot. Prefetched Address Translation. In *MICRO*, 2019.
- [109] Advanced Micro Devices. AMD-V Nested Paging, White Paper. <http://developer.amd.com/wordpress/media/2012/10/NPT-WP-1%201-final-TM.pdf>.
- [110] WikiChip. Intel Cascade Lake. https://en.wikichip.org/wiki/intel/cores/cascade_lake_sp.
- [111] Hyungiin Kim, Seongwook Kim, Junhyeok Park, Gwangeun Byeon, and Seokin Hong. Don't cache, speculate!: Speculative address translation for flash-based storage systems. *IEEE Access*, 2025.
- [112] OSDev Wiki. Page frame allocation. https://wiki.osdev.org/Page_Frame_Allocation. Accessed: 2025-08-18.
- [113] Anonymous Memory. <https://docs.kernel.org/admin-guide/mm/concepts.html>.
- [114] Konstantinos Kanellopoulos, Konstantinos Sgouras, Harsh Songara, Andreas Kosmas Kakolyris, Vlad-Petru Nitu, Spiros Galanopoulos, Rahul Bera, Konstantina Koliogeorgi, Rakesh Kumar, and Onur Mutlu. Revelator: Rapid Data Fetching via System-Software-Guided Hash-based Speculative Address Translation (Extended Version). 2026. arXiv.
- [115] Google. CITY Hash. <https://github.com/google/cityhash>.
- [116] Jovan Stojkovic, Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, and Josep Torrellas. Parallel Virtualized Memory Translation with Nested Elastic Cuckoo Page Tables. In *ASPLOS*, 2022.
- [117] Jovan Stojkovic, Namrata Mantri, Dimitrios Skarlatos, Tianyin Xu, and Josep Torrellas. Memory-Efficient Hashed Page Tables. In *HPCA*, 2023.
- [118] Faruk Guvenilir and Yale N Patt. Tailored Page Sizes. In *ISCA*, 2020.
- [119] Stratos Psomadakis, Chloe Alverti, Vasileios Karakostas, Christos Katsakioris, Dimitrios Siakavaras, Konstantinos Nikas, Georgios Goumas, and Nectarios Koziris. Elastic Translations: Fast Virtual Memory with Multiple Translation Sizes. In *MICRO*, 2024.
- [120] Chang Hyun Park, Taekyung Heo, Jungi Jeong, and Jaehyuk Huh. Hybrid TLB Coalescing: Improving TLB Translation Coverage Under Diverse Fragmented Memory Allocations. In *ISCA*, 2017.
- [121] Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, and Onur Mutlu. Mosaic: A GPU Memory

- Manager with Application-Transparent Support for Multiple Page Sizes. In *MICRO*, 2017.
- [122] The Linux Kernel Documentation. NUMA Memory Policy. https://www.kernel.org/doc/html/latest/admin-guide/mm/numa_memory_policy.html. Accessed: 2026-05-16.
 - [123] Michael Kerrisk. `set_mempolicy(2)`: Linux Manual Page. https://man7.org/linux/man-pages/man2/set_mempolicy.2.html. Linux man-pages 6.16, accessed: 2026-05-16.
 - [124] Michael Kerrisk. `mbind(2)`: Linux Manual Page. <https://man7.org/linux/man-pages/man2/mbind.2.html>. Linux man-pages 6.16, accessed: 2026-05-16.
 - [125] Xiangyao Yu, Christopher J Hughes, Nadathur Satish, and Srinivas Devadas. IMP: Indirect memory prefetcher. In *MICRO*, 2015.
 - [126] Sam Ainsworth and Timothy M Jones. An event-triggered programmable prefetcher for irregular workloads. 2018.
 - [127] Nishil Talati, Kyle May, Armand Behrooz, Y. Yang, Kuba Kaszyk, Christos Vasiladiotis, Tarunesh Verma, L. Li, Brandon Nguyen, Jiawen Sun, J. Morton, Agreen Ahmadi, T. Austin, M. O’Boyle, S. Mahlke, T. Mudge, and Ronald Drelinski. Prodigy: Improving the Memory Latency of Data-Indirect Irregular Workloads Using Hardware-Software Co-Design. In *HPCA 2021*.
 - [128] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramanian, Chris Wilkerson, Seth H. Pugsley, and Zeshan Chishti. Efficiently Prefetching Complex Address Patterns. In *MICRO*, 2015.
 - [129] Robert Cooksey, Stephan Jourdan, and Dirk Grunwald. A Stateless, Content-Directed Data Prefetching Mechanism. 2002.
 - [130] Eiman Ebrahimi, Onur Mutlu, and Yale N Patt. Techniques for bandwidth-efficient prefetching of linked data structures in hybrid prefetching systems. In *HPCA*, 2009.
 - [131] Amir Roth, Andreas Moshovos, and Gurindar S. Sohi. Dependence based prefetching for linked data structures. In *ASPLOS*, 1998.
 - [132] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading Kernel Memory From User Space. In *USENIX Security*, 2018.
 - [133] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. Netspectre: Read arbitrary memory over network. In *Computer Security—ESORICS 2019: 24th European Symposium on Research in Computer Security, Luxembourg, September 23–27, 2019, Proceedings, Part I 24*, pages 279–299. Springer, 2019.
 - [134] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B Lee. Last-level cache side-channel attacks are practical. In *2015 IEEE symposium on security and privacy*, pages 605–622. IEEE, 2015.
 - [135] David Kroft. Lockup-free instruction fetch/prefetch cache organization. In *ISCA*, 1981.
 - [136] Apache Cassandra. <https://cassandra.apache.org/>.
 - [137] MongoDB Recommends Disabling Huge Pages. <https://docs.mongodb.org/manual/tutorial/transparent-huge-pages/>, 2016.
 - [138] SQLite. <https://www.sqlite.org/>.
 - [139] Apache Spark. <https://spark.apache.org/>.
 - [140] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzynek, and Krste Asanović. Chisel: Constructing Hardware in a Scala Embedded Language. In *DAC*, 2012.
 - [141] Yosys open synthesis suite. <https://github.com/YosysHQ/>.
 - [142] Nangate freepdk45 open cell library. <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git>.
 - [143] Amer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In *ISCA*, 2010.
 - [144] Sam Ainsworth and Timothy M. Jones. Compendia: Reducing Virtual-Memory Costs Via Selective Densification. In *ISMM*, 2021.
 - [145] Chang Hyun Park, Ilias Vougioukas, Andreas Sandberg, and David Black-Schaffer. Every Walk’s a Hit: Making Page Walks Single-Access Cache Hits. In *ASPLOS*, 2022.
 - [146] Osang Kwon, Yongho Lee, Junhyeok Park, Sungbin Jang, Byungchul Tak, and Seokin Hong. Distributed Page Table: Harnessing Physical Memory as an Unbounded Hashed Page Table. In *MICRO*, 2024.
 - [147] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, David Brooks, and Gu-Yeon Wei. Workload Traces for Google Warehouse-Scale Computers. *IEEE Computer Architecture Letters*, 19(2):142–145, 2020.
 - [148] Michael A. Bender, Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once. In *J. ACM*, 2023.
 - [149] Intel Xeon Gold 6226R. https://en.wikichip.org/wiki/intel/xeon_gold/6226r.
 - [150] Hewlett Packard. McPAT. <https://github.com/HewlettPackard/mcpat>.
 - [151] Javier Picorel, Djordje Jevdjic, and Babak Falsafi. Near-Memory Address Translation. In *PACT*, 2017.
 - [152] A. J. Smith. A Comparative Study of Set Associative Memory Mapping Algorithms and Their Use for Cache and Main Memory. In *IEEE TSE*, 1978.
 - [153] Siyuan Chai, J Zhang, J Kim, A Wang, F Chung, et al. EMT: An OS framework for new memory translation architectures. In *OSDI*, 2025.
 - [154] Chang Hyun Park, Sanghoon Cha, Bokyeong Kim, Youngjin Kwon, David Black-Schaffer, and Jaehyuk Huh. Perforated Page: Supporting Fragmented Memory Allocation for Large Pages. In *ISCA*, 2020.
 - [155] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. Coordinated and Efficient Huge Page Management with Ingens. In *OSDI*, 2016.
 - [156] Madhusudhan Talluri, Shing Kong, Mark D. Hill, and David A. Patterson. Tradeoffs in Supporting Two Page Sizes. In *ISCA*, 1992.
 - [157] Ashish Panwar, Aravinda Prasad, and K Gopinath. Mak-

- ing Huge Pages Actually Useful. In *ASPLOS*, 2018.
- [158] Ashish Panwar, Sorav Bansal, and K Gopinath. Hawkeye: Efficient Fine-grained OS Support for Huge Pages. In *ASPLOS*, 2019.
 - [159] Venkat Sri Sai Ram, Ashish Panwar, and Arkaprava Basu. Trident: Harnessing Architectural Resources for All Page Sizes in X86 Processors. In *MICRO*, 2021.
 - [160] Zhen Fang, Lixin Zhang, J.B. Carter, W.C. Hsieh, and S.A. McKee. Reevaluating Online Superpage Promotion with Hardware Support. In *HPCA*, 2001.
 - [161] Mark Swanson, Leigh Stoller, and John Carter. Increasing TLB Reach Using Superpages Backed By Shadow Memory. In *ISCA*, 1998.
 - [162] Yu Du, Miao Zhou, Bruce R Childers, Daniel Mossé, and Rami Melhem. Supporting Superpages in Non-Contiguous Physical Memory. In *HPCA*, 2015.
 - [163] Madhusudhan Talluri and Mark D. Hill. Surpassing the TLB Performance of Superpages with Less Operating System Support. In *ASPLOS*, 1994.
 - [164] Mel Gorman and Patrick Healy. Supporting Superpage Allocation Without Additional Hardware Support. In *ISMM*, 2008.
 - [165] Mohammad Agbarya, Idan Yaniv, Jayneel Gandhi, and Dan Tsafir. Predicting Execution Times with Partial Simulations in Virtual Memory Research: Why and How. In *MICRO*, 2020.
 - [166] Narayanan Ganapathy and Curt Schimmel. General Purpose Operating System Support for Multiple Page Sizes. In *ATC*, 1998.
 - [167] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacherjee. Translation Ranger: Operating System Support for Contiguity-Aware TLBs. In *ISCA*, 2019.
 - [168] Dongwei Chen, Dong Tong, Chun Yang, Jiangfang Yi, and Xu Cheng. FlexPointer: Fast Address TranslatiOn Based On Range TLB and Tagged Pointers. In *TACO*, 2023.
 - [169] Binh Pham, Viswanathan Vaidyanathan, Aamer Jaleel, and Abhishek Bhattacherjee. CoLT: Coalesced Large-Reach TLBs. In *MICRO*, 2012.
 - [170] Binh Pham, Abhishek Bhattacherjee, Yasuko Eckert, and Gabriel H Loh. Increasing TLB Reach By Exploiting Clustering in Page Translations. In *HPCA*, 2014.
 - [171] Samira Mirbagher-Ajorpaz, Elba Garza, Gilles Pokam, and Daniel A. Jiménez. CHiRP: Control-Flow History Reuse Prediction. In *MICRO*, 2020.
 - [172] Misel-Myrto Papadopolou, Xin Tong, André Seznec, and Andreas Moshovos. Prediction-Based Superpage-Friendly TLB Designs. In *HPCA*, 2015.
 - [173] Mohan Kumar Kumar, Steffen Maass, Sanidhya Kashyap, Ján Veselý, Zi Yan, Taesoo Kim, Abhishek Bhattacherjee, and Tushar Krishna. Latr: Lazy Translation Coherence. In *ASPLOS*, 2018.
 - [174] Toni Juan, Tomas Lang, and Juan J. Navarro. Reducing TLB Power Requirements. In *ISLPED*, 1997.
 - [175] T.H. Romer, W.H. Ohlrich, A.R. Karlin, and B.N. Bershad. Reducing TLB and Memory Overhead Using Online Superpage Promotion. In *ISCA*, 1995.
 - [176] I. Kadayif, P. Nath, M. Kandemir, and A. Sivasubramaniam. Compiler-directed Physical Address Generation for Reducing dTLB Power. In *ISPASS*, 2004.
 - [177] Tianhao Zheng, Haishan Zhu, and Mattan Erez. SIPT: Speculatively Indexed, Physically Tagged Caches. In *HPCA*, 2018.
 - [178] D. A. Wood, S. J. Eggers, G. Gibson, M. D. Hill, and J. M. Pendleton. An In-Cache Address Translation Mechanism. In *ISCA*, 1986.
 - [179] A. Seznec. Concurrent Support of Multiple Page Sizes on a Skewed Associative TLB. In *TC*, 2004.
 - [180] Georgios Vavouliotis, Lluc Alvarez, Vasileios Karakostas, Konstantinos Nikas, Nectarios Koziris, Daniel A. Jiménez, and Marc Casas. Exploiting Page Table Locality for Agile TLB Prefetching. In *ISCA*, 2021.
 - [181] Georgios Vavouliotis, Lluc Alvarez, Boris Grot, Daniel Jiménez, and Marc Casas. Morrigan: A Composite Instruction TLB Prefetcher. In *MICRO*, 2021.
 - [182] Gokul B Kandiraju and Anand Sivasubramaniam. Going the Distance for TLB Prefetching: An Application-driven Study. In *ISCA*, 2002.
 - [183] Ashley Saulsbury, Fredrik Dahlgren, and Per Stenström. Recency-based TLB Preloading. In *ISCA*, 2000.
 - [184] Kavita Bala, M. Frans Kaashoek, and William E. Weihl. Software Prefetching and Caching for Translation Lookaside Buffers. In *OSDI*, 1994.
 - [185] Abhishek Bhattacherjee. Large-Reach Memory Management Unit Caches. In *MICRO*, 2013.
 - [186] Abhishek Bhattacherjee, Daniel Lustig, and Margaret Martonosi. Shared Last-Level TLBs for Chip Multiprocessors. In *ISCA*, 2011.
 - [187] Srikanth Bharadwaj, Guilherme Cox, Tushar Krishna, and Abhishek Bhattacherjee. Scalable Distributed Last-Level TLBs Using Low-Latency Interconnects. In *MICRO*, 2018.
 - [188] Dongrui Fan, Zhimin Tang, Hailin Huang, and G.R. Gao. An Energy Efficient TLB Design Methodology. In *ISLPED*, 2005.
 - [189] B. Pratheek, Neha Jawalkar, and Arkaprava Basu. Improving GPU Multi-tenancy with Page Walk Stealing. In *HPCA*, 2021.
 - [190] Yashwant Marathe, Nagendra Gulur, Jee Ho Ryoo, Shuang Song, and Lizy K. John. CSALT: Context Switch Aware Large TLB. In *MICRO*, 2017.
 - [191] Yunfang Tai, Wanwei Cai, Qi Liu, Ge Zhang, and Wenzhi Wang. Comparisons of Memory Virtualization Solutions for Architectures with Software-Managed TLBs. In *NAS*, 2013.
 - [192] Xiaotao Chang, Hubertus Franke, Yi Ge, Tao Liu, Kun Wang, Jimi Xenidis, Fei Chen, and Yu Zhang. Improving Virtualization in the Presence of Software Managed Translation Lookaside Buffers. In *ISCA*, 2013.
 - [193] Bruce L. Jacob and Trevor N. Mudge. A Look at Several Memory Management Units, TLB-Refill Mechanisms, and Page Table Organizations. In *ASPLOS*, 1998.
 - [194] D. R. Cheriton, G. A. Slavenburg, and P. D. Boyle. Software-Controlled Caches in the VMP Multiprocessor. In *ISCA*, 1986.
 - [195] David Nagle, Richard Uhlig, Tim Stanley, Stuart Sechrest, Trevor N. Mudge, and Richard B. Brown. Design Trade-offs for Software-managed TLBs. In *ISCA*, 1993.
 - [196] Chandrashis Mazumdar, Prachatos Mitra, and

- Arkaprava Basu. Dead Page and Dead Block Predictors: Cleaning TLBs and Caches Together. In *HPCA*, 2021.
- [197] Jagadish B. Kotra, Michael LeBeane, Mahmut T. Kandemir, and Gabriel H. Loh. Increasing GPU Translation Reach by Leveraging Under-Utilized On-Chip Resources. In *MICRO*, 2021.
- [198] Aamer Jaleel, Eiman Ebrahimi, and Sam Duncan. DUCATI: High-Performance Address Translation by Extending TLB Reach of GPU-Accelerated Systems. In *TACO*, 2019.